An-Najah National University

Shield Gates

Secure Email System with Mail Gateway Filtering

Technical Documentation

The Supervisor:

Dr. Othman M. Othman & A. Dr. Prof Jehad M. Hamamreh

By:

Rama Abu-Haweleha	12219068
Rana Shiqer	12218105

Graduation Project

An-Najah National University

Faculty of Engineering and Information Technology

Department of Computer Science

Networks and Information Security fields

Academic Year 2025-2026/Second Semester

Dedication

We dedicate this work to Palestine, and especially to Gaza, whose resilience, courage, and unwavering spirit have inspired us throughout this journey. Despite immense challenges, Gaza has shown the world that hope, determination, and the pursuit of knowledge can endure even in the most difficult circumstances.

We also dedicate this achievement to our families, whose love, support, patience, and sacrifices have been the foundation of our success. Their encouragement has motivated us to persevere and strive for excellence.

Our sincere gratitude goes to our supervisors, Dr. Othman Othman and Dr. Jihad Hamamreh, for their invaluable guidance, support, and encouragement throughout this project. We also extend our appreciation to the members of the examination committee for their time, expertise, and constructive feedback.

To our professors, colleagues, and friends, thank you for your support, inspiration, and the memorable experiences we shared throughout our academic journey.

This work represents not only the completion of a project but also the culmination of years of learning, perseverance, and growth. We hope it reflects the knowledge and values we have gained and serves as a meaningful step toward our future aspirations.

1. Abstract

This report presents the design and implementation of a secure email system utilizing a specialized mail gateway for enhanced filtering. The system acts as an intermediary gateway positioned between external clients and the primary mail server to mitigate threats from malicious actors. Key protective features include the detection of sensitive information requests, malicious URLs, and suspicious PDF attachments. Furthermore, the gateway provides email spooling capabilities to ensure message retention during server downtime and employs techniques to bypass traditional firewall and intrusion detection system (IDS) limitations. The filtering process is organized into three distinct stages: header authentication risk assessment using SPF, DKIM, and DMARC; body text analysis leveraging natural language processing (NLP) models such as BERT and RoBERTa; and deep attachment analysis including entropy checks and sandboxing. A central decision engine aggregates the resulting risk scores to determine whether to drop or deliver each message. The implementation utilizes a technology stack comprising Postfix, Dovecot, Thunderbird, VirtualBox, and Ubuntu virtual machines.

Table of Contents:

1. Introduction

Email remains a primary communication tool for both personal and professional interactions, making it a frequent target for cyberattacks. Organizations face a continuous influx of malicious emails containing phishing attempts, malware-laden attachments, and sophisticated social engineering tactics. Traditional perimeter defenses, such as standard firewalls and Intrusion Detection Systems (IDS), often struggle to keep pace with evolving email-based threats that exploit application-layer vulnerabilities.

The "Secure Email System with Mail Gateway Filtering" project addresses these challenges by introducing a dedicated gateway layer. This gateway serves as a proactive filter that scrutinizes incoming traffic before it reaches the internal mail infrastructure. By isolating the mail server from direct exposure to the outer internet, the system reduces the attack surface and provides a centralized point for advanced threat detection and policy enforcement.

2.1. Problem Statement

Modern email threats have evolved beyond simple spam, requiring more sophisticated detection methods. The following core problems are addressed by the proposed system:

- Sensitive Information Requests: Phishing attacks often attempt to deceive users into revealing credentials or confidential data through urgent or deceptive requests.
- Malicious URLs and Attachments: Emails frequently carry links to malicious websites or attachments (such as PDFs and JavaScript files) designed to execute unauthorized code or install malware.
- System Availability: When a mail server experiences downtime, incoming emails may be lost or bounced back to the sender, leading to communication gaps.
- Detection Evasion: Attackers continuously develop techniques to bypass standard security measures, necessitating a multi-layered approach that includes behavioral and structural analysis.

2.2. Objectives

The primary objective of this project is to develop a robust email security gateway with the following specific goals:

1. Intermediate Protection: Implement a gateway that intercepts traffic between external clients and the internal mail server to filter malicious intent.
2. Multi-Layered Risk Scoring: Establish a three-stage detection pipeline covering headers, body content, and attachments to produce comprehensive risk scores.
3. Advanced NLP Analysis: Utilize state-of-the-art models like BERT and RoBERTa to identify phishing intent and urgency in message text.
4. Safe Execution Environment: Incorporate sandboxing techniques to analyze potentially harmful attachments and installers without risking the host system.
5. Spooling and Redundancy: Provide spooling mechanisms to store emails temporarily when the primary mail server is unavailable.
6. Integration of Open-Source Tools: Leverage established technologies such as Postfix, Dovecot, and Ubuntu for a cost-effective and scalable solution.

2. System Architecture

The architecture of the Secure Email System is designed as a modular gateway that sits between the external internet (outer clients) and the internal mail server.

3.1 Structural Overview

The system is hosted on Ubuntu virtual machines within a VirtualBox environment. The gateway intercepts all incoming SMTP traffic. The internal mail server runs Postfix for mail transfer and Dovecot for mail retrieval, while Thunderbird serves as the end-user mail client.

3.2 Component Interaction

1. Mail Gateway: The entry point for all external email. It hosts the three-stage detection pipeline and the decision engine.
2. Internal Mail Server: Receives filtered emails from the gateway and handles local delivery via Dovecot.
3. Spooling Module: An integrated part of the gateway that queues messages if the internal server is unreachable.

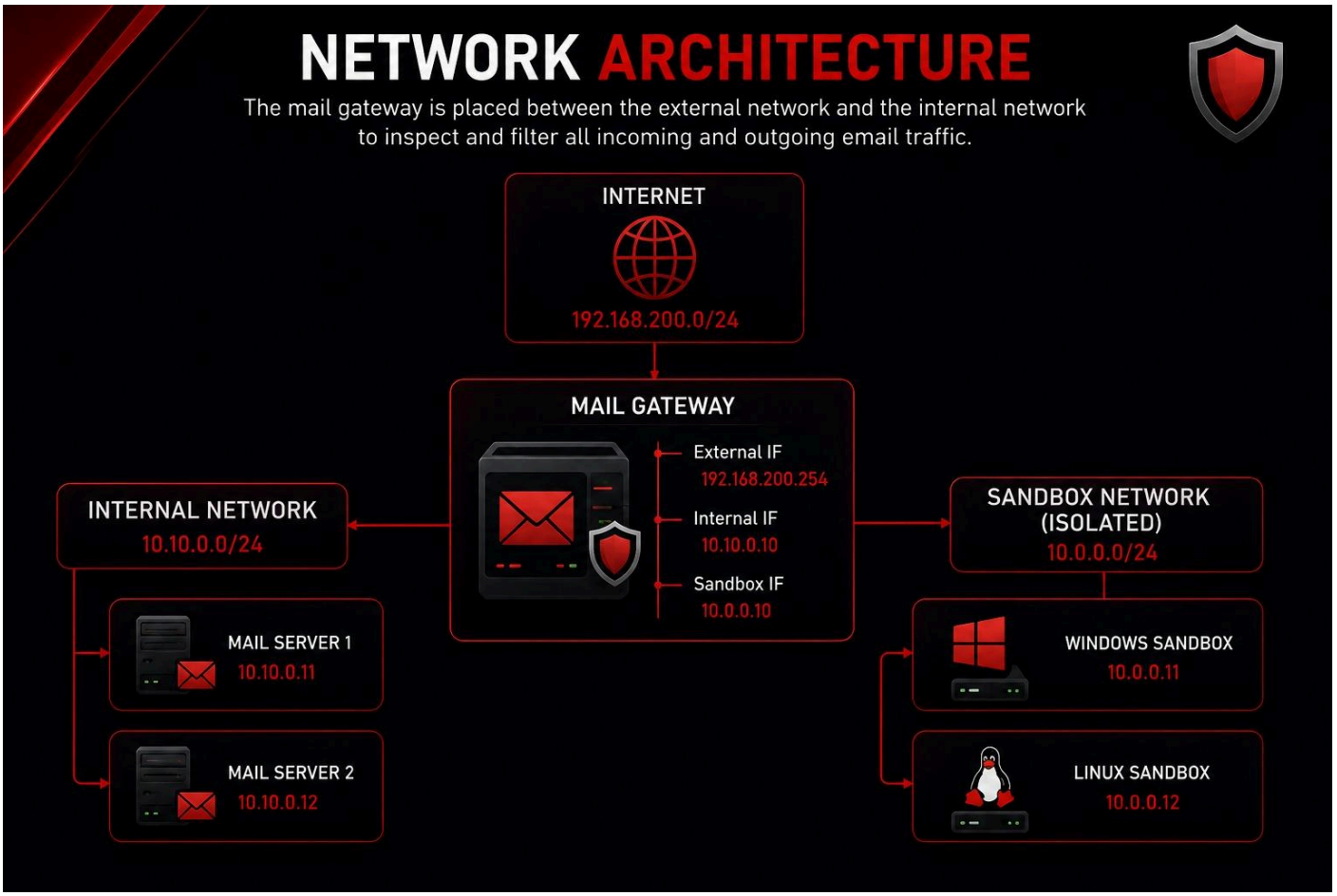


Figure 1

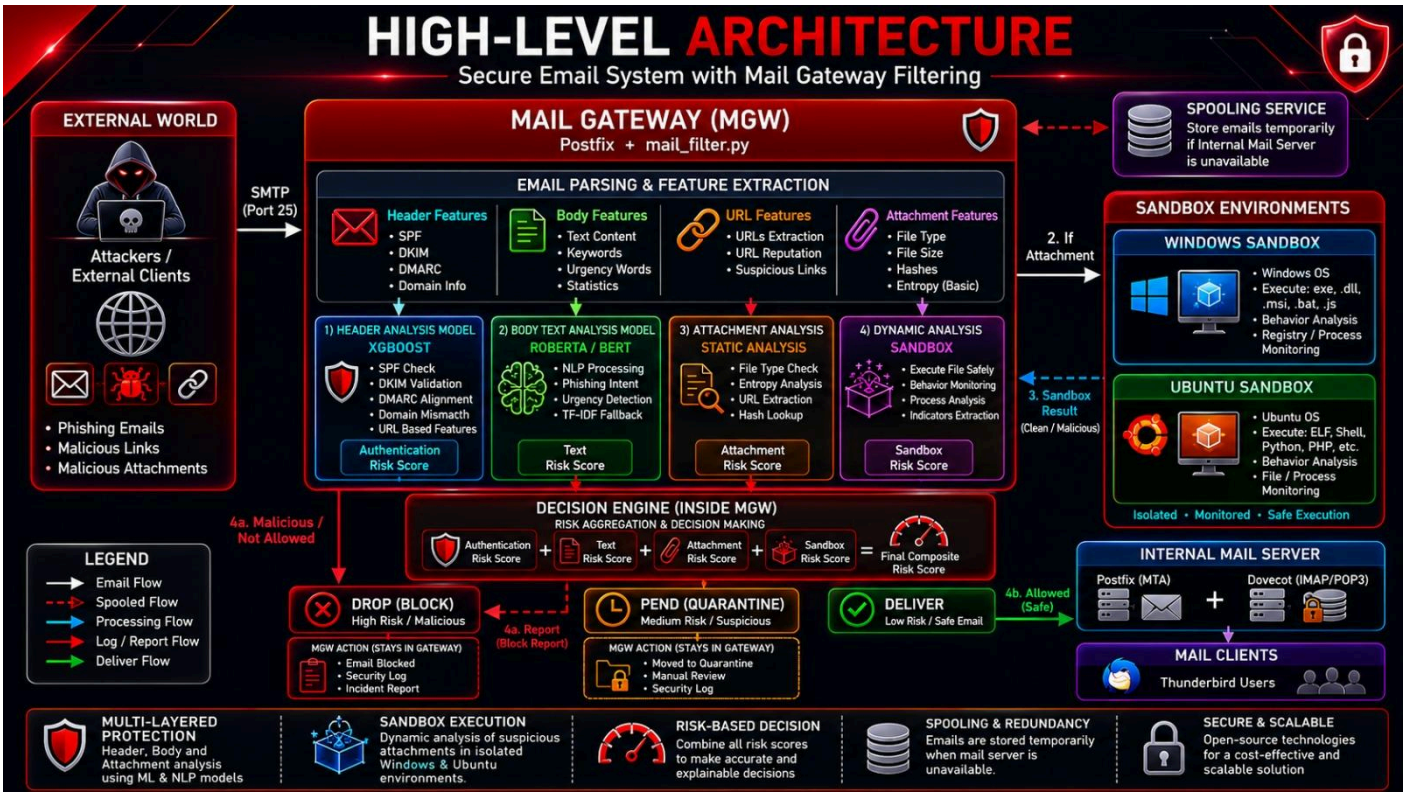


Figure 2:

4. Related Works

Existing commercial solutions provide benchmarks for email security gateways. Two prominent systems are Proofpoint Email Protection and Cisco Secure Email Gateway.

4.1. Proofpoint Email Protection

Proofpoint Email Protection functions as an anti-spam appliance designed to capture and classify malicious email objects, including spam, phishing, and malware [1]. It employs classification-based mechanisms to filter content and analyze behavior. The system is evaluated using confusion-matrix metrics to ensure detection accuracy across various categories, such as spoofing, impostor messages, and bulk email [1]. By utilizing these classification metrics, Proofpoint can effectively manage and mitigate a wide range of email-based threats.

4.2. Cisco Secure Email Gateway

While Cisco Secure Email Gateway is a recognized solution in the industry, the provided literature does not contain sufficient evidence to describe its specific filtering mechanisms or operational behavior. Consequently, its technical details are not available in the current sources.

5. Methodology and Procedure

The system employs a sequential multi-stage analysis process to evaluate the risk of each incoming email.

5.1 Stage 1: Header Detection

This stage focuses on validating the authenticity of the sender and the integrity of the email headers.

- Ml model: XGBoost is an ensemble of decision trees for feature engineering and pattern recognition.
- Protocols: SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail), and DMARC (Domain-based Message Authentication, Reporting, and Conformance) are verified to detect spoofing.
- Domain Validation: Checks for "look-alike" domains (typosquatting) and anomalous header fields.
- Output: The stage generates an authentication risk score.

5.2. Stage 2: Body Text NLP

The content of the email is analyzed using deep learning models to understand intent.

- Models: BERT and RoBERTa are utilized to process the message body.
- Detection Goals: The models identify phishing markers, malicious intent, and excessive urgency, which are common in social engineering.
- Output: The stage generates a Text Risk Score.

5.3. Stage 3: Attachment Analysis

Attachments are scrutinized to identify hidden threats.

3. Static Analysis: Checks for URL reputation within files and calculates file entropy to detect packed or encrypted malware.
4. Dynamic Analysis (Sandbox): Suspicious files, such as JavaScript (JS) files or installers, are executed in a controlled sandbox environment to observe their behavior.
5. Output: The stage generates an attachment risk score.

5.4. Decision Engine

The Decision Engine receives the three scores (Authentication, Text, and Attachment). It applies a combining logic—such as a weighted average or a threshold-based rule set—to decide the final action. If the cumulative risk exceeds a predefined limit, the email is dropped; otherwise, it is delivered to the internal mail server

6. Methodology & Procedure

6.1 VMs Network configuration

Before network configuration, we need to verify the network interfaces on each machine. Can use one of the following cmd:

- `ip link show`
- `ip addr show`

1. On all VMs, we need to edit the netplan configuration and configure all interfaces, setting each one in a different network in case MGW

   ```
   sudo nano /etc/netplan/00-installer-config.yaml
   ```

And after editing the netplan by setting the ip address and setting a static route, apply the netplan configuration with `"sudo netplan apply."` In figs. 6.6.1-4, the netplan content and the network files are shown, which may be different from one VM to another.

```
rama@mgw:~/Desktop$ ls /etc/netplan/
01-network-manager-all.yaml
rama@mgw:~/Desktop$ sudo nano /etc/netplan/01-network-manager-all.yaml
[sudo] password for rama:
rama@mgw:~/Desktop$ sudo cat /etc/netplan/01-network-manager-all.yaml
# Let NetworkManager manage all devices on this system
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.200.254/24
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
      routes:
        - to: default
          via: 192.168.200.1
    enp0s8:
      dhcp4: no
      addresses:
        - 10.10.0.10/24
rama@mgw:~/Desktop$
```

Fig. 6.1.1: netplan files on mgw

```
rama@mail:~/Desktop$ ls /etc/netplan
01-network-manager-all.yaml
rama@mail:~/Desktop$ sudo nano /etc/netplan/01-network-manager-all.yaml
[sudo] password for rama:
rama@mail:~/Desktop$ sudo cat /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 10.10.0.20/24
      routes:
        - to: default
          via: 10.10.0.10
      nameservers:
        addresses: [8.8.8.8]
rama@mail:~/Desktop$
```

fig. 6.1.2 netplan files on mgw

```
rama@rama:~/Desktop$ sudo cat /etc/netplan/01-network-manager-all.yaml
[sudo] password for rama:
# /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 10.10.0.30/24
      routes:
        - to: default
          via: 10.10.0.10
        - to: 192.168.200.0/24
          via: 10.10.0.10
      nameservers:
        addresses: [8.8.8.8]
```

Fig. 6.1.3: client inside the internal network 10.10.0.0/24

```
rama@rama:~/Desktop$ sudo cat /etc/netplan/01-network-manager-all.yaml
[sudo] password for rama:
# /etc/netplan/01-network-manager-all.yaml
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.200.100/24
      routes:
        - to: default
          via: 192.168.200.1
        - to: 10.10.0.0/24
          via: 192.168.200.254
      nameservers:
        addresses: [8.8.8.8]
```

Fig. 6.1.4: client inside NatNetwork 192.168.200.0/24

2. **In the fig. 6.1.5, we use list of commands that will help us to Enable IP Forwarding and Routing**

```
rama@mgw:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
rama@mgw:~$ echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
net.ipv4.ip_forward=1
rama@mgw:~$ sudo sysctl -p
net.ipv4.ip_forward = 1
net.ipv4.ip_forward = 1
rama@mgw:~$
```

Fig. 6.1.5: enable ip forwarding

3. **And the commands used in figures 6.1.6.a and b are used to configure NAT/Masquerading on MGW. Enable NAT so the internal network can communicate externally (if needed):**

```
rama@mgw:~$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://ps.archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://ps.archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://ps.archive.ubuntu.com/ubuntu focal-backports InRelease [128 kB]
Get:5 http://security.ubuntu.com/ubuntu focal-security/main amd64 DEP-11 Metadata [74.7 kB]
Get:6 http://security.ubuntu.com/ubuntu focal-security/restricted amd64 DEP-11 Metadata [208 B]
Get:7 http://security.ubuntu.com/ubuntu focal-security/universe amd64 DEP-11 Metadata [160 kB]
Get:8 http://ps.archive.ubuntu.com/ubuntu focal-updates/main amd64 DEP-11 Metadata [276 kB]
Get:9 http://security.ubuntu.com/ubuntu focal-security/multiverse amd64 DEP-11 Metadata [940 B]
Get:10 http://ps.archive.ubuntu.com/ubuntu focal-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:11 http://ps.archive.ubuntu.com/ubuntu focal-updates/universe amd64 DEP-11 Metadata [445 kB]
Get:12 http://ps.archive.ubuntu.com/ubuntu focal-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:13 http://ps.archive.ubuntu.com/ubuntu focal-backports/main amd64 DEP-11 Metadata [7,976 B]
Get:14 http://ps.archive.ubuntu.com/ubuntu focal-backports/restricted amd64 DEP-11 Metadata [212 B]
Get:15 http://ps.archive.ubuntu.com/ubuntu focal-backports/universe amd64 DEP-11 Metadata [30.5 kB]
Get:16 http://ps.archive.ubuntu.com/ubuntu focal-backports/multiverse amd64 DEP-11 Metadata [212 B]
Fetched 1,380 kB in 2s (897 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
132 packages can be upgraded. Run 'apt list --upgradable' to see them.
rama@mgw:~$
rama@mgw:~$ sudo apt install iptables iptables-persistent -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfprint-2-tod1 libllvm10
```

Fig. 6.1.6.a: Install iptables if not present

```
rama@mgw:~$ sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE
rama@mgw:~$
rama@mgw:~$ sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT
rama@mgw:~$
rama@mgw:~$ sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state RELATED,ESTABLISHED -j ACCEPT
rama@mgw:~$
rama@mgw:~$ sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save
```

Fig. 6.1.6.b: configure NAT & save rules

4. We need to update **`/etc/hosts`** all VMs to resolve the hostnames of the mail server & gateway, as shown in figs. 6.1.7

```
rama@mgw:~/Desktop$ sudo cat /etc/hosts
[sudo] password for rama:
127.0.0.1 localhost

192.168.200.254 mgw.company.com mgw
10.10.0.20 mail.company.com mail
10.10.0.30 employee.company.com Employee
```

```
rama@mail:~/Desktop$ sudo cat /etc/hosts
[sudo] password for rama:
127.0.0.1 localhost

10.10.0.20 mail.company.com mail
10.10.0.10 mgw.company.com mgw
```

```
rama@rama:~/Desktop$ sudo cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       rama

10.10.0.10 mgw.company.com mgw
192.168.200.254 mgw.company.com mgw
10.10.0.20 mail.company.com mail
```

Fig. 6.1.7: DNS "hosts name" configuration

This change to the network configuration uses a hybrid IP addressing strategy that includes both a DHCP-assigned address (192.168.200.5) and a static IP address (192.168.200.100). The static IP makes sure that service endpoints are always reliable and stable, while the DHCP address helps with connectivity during migration by providing redundancy. The configuration also improves routing by adding custom static routes and metric-based path selection. This lets you access the 10.10.0.0/24 network directly through gateway 192.168.200.254. This setup makes traffic flows better and can handle failover situations. Using Google's 8.8.8.8 servers for an independent DNS configuration makes sure that names are always resolved correctly. This makes the network more reliable and makes it easier to fix problems.

6.2 Mail Service Infrastructure

On our system and to test the validity of our system and the email message flow, we will implement the mail service infrastructure by using Postfix & Dovecot for mail & delivery services.

Before starting any configuration, we need to Install the required packages on the mail server machine by using the command shown in fig. 6.2.1, and while installing the postfix, choose `Internet Site`, and write the domain `company.com`

```
rama@mail:~$ sudo apt install postfix dovecot-imapd dovecot-pop3d
```

Fig. 6.2.1: required packaged on mail server

6.1.1 Dovecote (High-Performance Mail Access)

Dovecot serves as both an MDA and a complete IMAP/POP3 server, focused on memory efficiency and transactional mailbox handling. It utilizes per-mailbox index files to significantly improve folder listing times and supports various mailbox formats, including mbox, Maildir, and the proprietary mdbox format. Security measures incorporate privilege separation and diverse authentication methods, alongside protections against brute-force attacks. However, the mbox format's non-standard nature and Dovecot's complex configuration could pose challenges for backups and configuration integrity.

Dovecot is typically not used on the filtering gateway; instead, it is configured on the internal mail server where emails are stored. Its role is to manage users.

mailboxes and provide access via IMAP or POP3, allowing users to read and manage their emails. In other words, Dovecot handles authentication and mailbox access, not filtering or mail routing.

The next flow steps to configure Dovecot in mail server

1. Fig. 6.2.2, showing how we create an SSL/TLS Certificate for Mail Server.

   To secure email communication, an SSL/TLS certificate and private key must be generated. This is typically done using the OpenSSL tool.

```
Help
ail:~/Desktop$ sudo openssl req -new -x509 -days 365 -nodes -out /etc/ssl/certs/mail.compa
n.crt -keyout /etc/ssl/private/mail.company.com.key
Generating a RSA private key
.....................................................................................
.....................................+++++
.....................................................+++++
writing new private key to '/etc/ssl/private/mail.company.com.key'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:company.com
Email Address []:
rama@mail:~/Desktop$ sudo chmod 600 /etc/ssl/private/mail.company.com.key
rama@mail:~/Desktop$ sudo chown root:root /etc/ssl/private/mail.company.com.key
```

Fig. 6.2.2: Generate self-signed SSH certificate

2. Figs. 6.2.3 show segment code in the main configuration file `"/etc/dovecot/dovecot.conf"` for dovecot. The configuration shown in Fig. 6.2, enables **Dovecot** to provide IMAP, POP3, and LMTP services on all network interfaces; stores user emails in Maildir format inside each user's home directory; and activates SSL/TLS encryption using the specified certificate and private key to secure email access and communication.

```
protocols = imap pop3 lmtp
listen = *

# Mail location
mail_location = maildir:~/Maildir

# SSL
ssl = yes
ssl_cert = </etc/ssl/certs/mail.company.com.crt
ssl_key = </etc/ssl/private/mail.company.com.key
```

Fig. 6.2.3: main dovecote config

3. **Inside Fig. 6.2.4, we add a configuration in Dovecot `"/etc/dovecot/conf.d/10-auth.conf"` that allows authentication using plain-text methods (PLAIN and LOGIN mechanisms) and permits users to authenticate without requiring encrypted connections, meaning login credentials can be sent in an unencrypted form if SSL/TLS is not enforced.**

```
disable_plaintext_auth = no
auth_mechanisms = plain login
```

Fig. 6.2.4: Dovecot Authentication Configuration and Plaintext Login Policy

4. **The configuration in Fig 6.2.5, defines how authentication services are exposed through Unix domain sockets inside `"/etc/dovecot/conf.d/10-master.conf."` The `auth-userdb` listener is reserved for internal user database communication, while the Postfix SMTP authentication socket `"/var/spool/postfix/private/auth"` enables secure communication between Postfix and Dovecot for SMTP user authentication. The permissions are set to allow the Postfix service to access the authentication socket securely, ensuring proper user login validation during email submission.**

```
  unix_listener auth-userdb {
    #mode = 0666
    #user =
    #group =
  }

  # Postfix smtp-auth
    unix_listener /var/spool/postfix/private/auth {
    mode = 0660
    user = postfix
    group = postfix
  }

  # Auth process is run as this user.
  #user = $default_internal_user
}
```

Fig. 6.2.5 Postfix Authentication Integration with Dovecot via Unix Sockets

5. **After doing our dovecot configuration, fig. 6.2.6 restarts the service to apply configuration changes and then checks whether the service is running correctly.**

```
rama@mail:~/Desktop$ sudo systemctl restart dovecot
rama@mail:~/Desktop$ sudo systemctl status dovecot
● dovecot.service - Dovecot IMAP/POP3 email server
     Loaded: loaded (/lib/systemd/system/dovecot.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-04-08 01:45:12 IDT; 6s ago
       Docs: man:dovecot(1)
             http://wiki2.dovecot.org/
   Main PID: 3069 (dovecot)
      Tasks: 4 (limit: 4577)
     Memory: 3.1M
     CGroup: /system.slice/dovecot.service
             ├─3069 /usr/sbin/dovecot -F
             ├─3071 dovecot/anvil
             ├─3072 dovecot/log
             └─3073 dovecot/config

Apr 08 01:45:12 mail.company.com systemd[1]: Started Dovecot IMAP/POP3 email server.
Apr 08 01:45:12 mail.company.com dovecot[3069]: master: Dovecot v2.3.7.2 (3c910f64b) starting
lines 1-16/16 (END)
```

Fig. 6.2.6: Managing and Verifying Dovecot Service Status

6.1.1 Postfix (Security by Design)

Postfix features a modular, multi-process architecture where core functions like queue management and SMTP reception run in separate, unprivileged processes, promoting security and damage containment. Sub-processes operate with reduced privileges and within chroot jails. Its design allows for high performance, with the ability to process over 1 million messages per hour using an efficient single-queue system and supporting modern protocols such as TLS 1.3 and

SMTPUTF8. However, it lacks native IMAP/POP3 support and has a configuration language that poses challenges for users familiar with monolithic MTAs.

Postfix is the core component you configure in an email filtering gateway because it handles all SMTP traffic, receiving incoming emails, applying filtering rules (such as spam or attachment checks), and then forwarding clean messages to the internal mail server. In this role, Postfix acts as a secure relay and control point for email flow, without storing user mailboxes or providing direct access to users.

Figure 6.4.7, shown The `/etc/postfix/main.cf` file, which is the core configuration for the Postfix mail server within the Secure Email Gateway (MGW). It dictates the handling of emails by defining system identity, network listening settings, and the relay mechanism. Postfix acts as a relay gateway, processing mail for the domain `company.com`, while delegating final delivery to an internal server. A key feature is the content filtering pipeline that redirects incoming emails to a custom filtering service via `content_filter = smtp:[127.0.0.1]:10025` for ML-based detection before delivery. Additionally, SASL authentication is managed by an external Dovecot service, ensuring secure user authentication without focusing on TLS encryption, which is disabled. The configuration enforces strict mail acceptance rules, allowing only trusted users to relay messages, thus reinforcing the security posture of the gateway as a filtering mechanism for internal mail. Overall, Postfix is transformed into a security-aware mail gateway, intercepting all emails for custom AI-based processing.

```
# ── Identity ──────────────────────────────────────────────────────────
myhostname    = mgw.company.com
mydomain      = company.com
myorigin      = $mydomain

smtpd_banner        = $myhostname ESMTP $mail_name (Ubuntu)
biff                = no
append_dot_mydomain = no
readme_directory    = no
compatibility_level = 2

alias_maps          = hash:/etc/aliases
alias_database      = hash:/etc/aliases
recipient_delimiter = +

# ── Network ───────────────────────────────────────────────────────────
inet_interfaces     = all
inet_protocols      = ipv4

# ── Domains ───────────────────────────────────────────────────────────
# MGW does NOT accept final delivery — it only relays
mydestination       = localhost
mynetworks          = 127.0.0.0/8, 192.168.200.0/24, 10.10.0.0/24
relay_domains       = company.com

# ── Relay outbound mail to the internal mail server ──────────────────
transport_maps      = hash:/etc/postfix/transport
relayhost           =

# ── Mailbox — MGW does not do local delivery ─────────────────────────
mailbox_size_limit = 0
message_size_limit = 52428800

# ── SASL Authentication (Dovecot SASL via TCP to mail server) ────────
# MGW authenticates clients but delegates SASL to mail.company.com's Dovecot
smtpd_sasl_type              = dovecot
smtpd_sasl_path              = inet:10.10.0.20:12345
smtpd_sasl_auth_enable       = yes
smtpd_sasl_security_options  = noanonymous
broken_sasl_auth_clients     = yes
```

Fig. 6.4.7a: postfix main configuration 1

```
# ── TLS ──────────────────────────────────────────────────────────────────
# TLS disabled for internal lab
smtpd_tls_security_level = none
smtp_tls_security_level  = none
smtpd_tls_auth_only      = no


#smtpd_tls_cert_file      = /etc/ssl/certs/mgw.company.com.crt
#smtpd_tls_key_file       = /etc/ssl/private/mgw.company.com.key
#smtpd_tls_security_level = may
#smtpd_tls_loglevel       = 1
#smtp_tls_security_level  = may

# ── Allow AUTH over plain connection (no TLS lab environment)────────────
smtpd_tls_auth_only = no

# ── Restrictions ─────────────────────────────────────────────────────────
smtpd_recipient_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_unauth_destination

smtpd_relay_restrictions =
    permit_mynetworks,
    permit_sasl_authenticated,
    reject_unauth_destination
```

Fig. 6.4.7b: postfix main configuration 2

The Postfix `master.cf` is configured as in fig. 6.4.8 with a standard SMTP service on port 25 and a submission service on port 587 for authenticated clients such as Thunderbird. The submission service is secured with SASL authentication via a Dovecot backend and routes outgoing mail through a content filter (`smtp:[127.0.0.1]:10025`) before final processing. After filtering, cleaned messages are re-injected into Postfix through a dedicated internal listener on `127.0.0.1:10026`, which accepts only trusted internal networks and bypasses additional content filtering to prevent loops. This design forms a multi-layer mail processing pipeline where incoming mail is authenticated, filtered by an external Python-based filter (`mail_filter.py`), and then safely reintroduced for delivery. Overall, the configuration implements a structured mail gateway architecture with clear separation between submission, filtering, and reinjection stages.

```
# ── SMTP port 25 ──────────────────────────────────────────────────────
smtp      inet  n       -       y       -       -       smtpd

# ── Submission port 587 – Thunderbird connects here ──────────────────
submission inet n       -       y       -       -       smtpd
  -o syslog_name=postfix/submission
  -o smtpd_tls_security_level=none
  -o smtpd_sasl_auth_enable=yes
  -o smtpd_sasl_type=dovecot
  -o smtpd_sasl_path=inet:10.10.0.20:12345
  -o smtpd_recipient_restrictions=permit_mynetworks,permit_sasl_authenticated,reject
  -o smtpd_relay_restrictions=permit_mynetworks,permit_sasl_authenticated,reject
  -o content_filter=smtp:[127.0.0.1]:10025
  -o receive_override_options=no_address_mappings
  -o milter_macro_daemon_name=ORIGINATING


# ── Re-inject port 10026 – mail_filter.py sends clean mail here ──────
# ── Re-inject port 10026 – mail_filter.py sends clean mail here ──────
127.0.0.1:10026 inet n  -       n       -       -       smtpd
  -o syslog_name=postfix/10026
  -o content_filter=
  -o receive_override_options=no_unknown_recipient_checks
  -o mynetworks=127.0.0.0/8,10.10.0.0/24,192.168.200.0/24
  -o smtpd_recipient_restrictions=permit_mynetworks,reject
  -o smtpd_relay_restrictions=permit_mynetworks,reject
```

Fig. 6.4.8: master file configuration

6.3 Mail Client Creation

1. On the mail server, we need to create a client mail account, which is adding a new server for each client, and assign it his mail directory, which represents the mail account, which will handle the mail messages. As a prepared step, I have already set up three accounts for three users, "mailnet1," "mailnet2," and "employee"; in fig. 6.3.1, I will set up a new account, "mailnet3," and assign his maildir.

```
rama@mail:~$ sudo adduser mailnet3
Adding user `mailnet3' ...
Adding new group `mailnet3' (1003) ...
Adding new user `mailnet3' (1003) with group `mailnet3' ...
Creating home directory `/home/mailnet3' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for mailnet3
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
rama@mail:~$ sudo maildirmake /home/mailnet3/Maildir
sudo: maildirmake: command not found
rama@mail:~$
rama@mail:~$ sudo mkdir -p /home/mailnet3/Maildir
rama@mail:~$ sudo mkdir /home/mailnet3/Maildir/{new,cur,tmp}
```

Fig. 6.3.1: create user account in mail server

2. **Now, we can switch to one of the client machines to prepare the mail service. start by installing required packages by using the command in fig. 6.3.2**

```
rama@rama:~$ sudo apt install mailutils msmtp msmtp-mta thunderbird -y
```

Fig. 6.3.2: mail client required packages

3. **Inside the Thunderbird app, we use the setting shown in figs. 6.3.3 & 4 to configure the incoming & outgoing servers.**

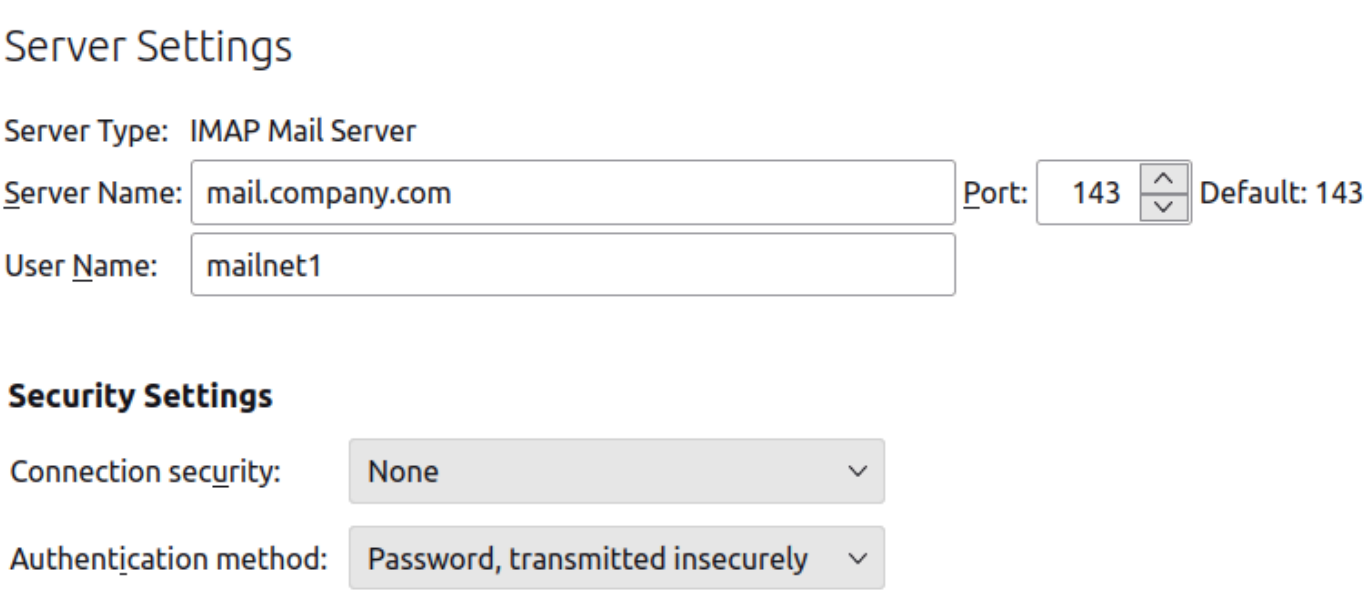


Fig. 6.3.3: IMAP incoming server setting

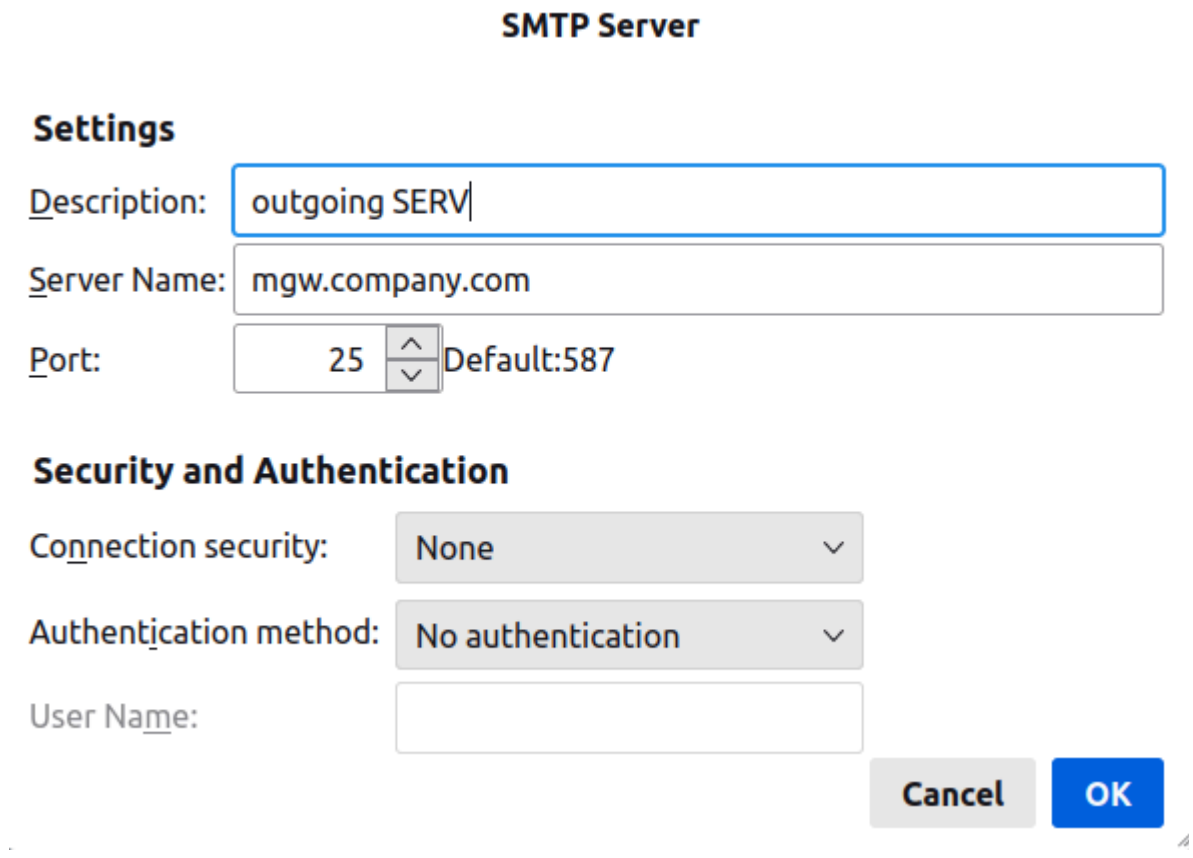


Fig. 6.3.4: SMTP outgoing server setting

6.4 System models

Our email filtering system, designed to prevent and block phishing and fraud emails, utilizes multiple interconnected and integrated modules. Each module employs various techniques and methods, enabling the system to identify and block phishing, fraud, and social engineering emails before they reach the email server.

The core concept involves using an evaluation system within each module to determine the final decision. Each module processes specific aspects and characteristics of the email, calculating the risk level probability based on all the analyzed features. The decision engine then aggregates the risk probabilities from all the modules to determine the overall risk level and decide whether to forward the email to the designated email server or drop it. The goal of this approach is to minimize the false positive probability.

We built the various system modules using machine learning models, natural language processing, and Python codes

The diagram 1 outlines the project structure, each part of which we will explain. The final code can be found in the GitHub repository [2].

Fig. 6.4.1 shows the configuration enables a **Postfix content filtering pipeline** using an external filter service.

The line `content_filter = smtp:[127.0.0.1]:10025` tells Postfix to send every incoming message (after initial SMTP acceptance) to a local SMTP-based filter running on port **10025**, which is typically your Python filter service (`mail_filter.py`). This allows the system to inspect, analyze, or modify emails before they continue through the delivery process. The setting `receive_override_options = no_address_mappings` disables alias and virtual address rewriting at this stage, ensuring the message is passed to the filter with its original addressing information intact, which is important for accurate analysis and feature extraction in your filtering system. Overall, this setup integrates Postfix with an external mail-processing layer, forming a central part of your mail gateway's inspection and classification workflow.

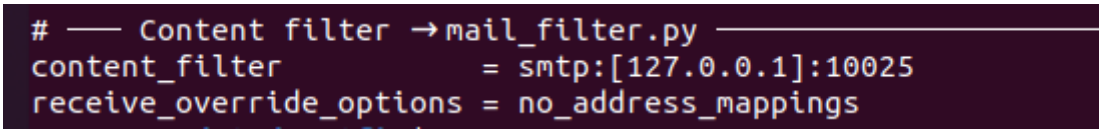


Fig. 6.4.1: activate content filtering with python

6.3.1 main filter layer

Previously in figs. 6.4.1 & 6.4.2, we set up an email filter content layer, and as in fig. 6.4.2, we interrupt the emails and redirect them into `~/MGW/mail_filter.py` codes.

The `mail_filter.py` script functions as the central controller of the Secure Email Gateway system, acting as an intelligent SMTP proxy that intercepts incoming email messages, analyzes them, and determines whether they should be delivered or blocked. When an email is received, it is first parsed using Python's email processing libraries to extract its headers and body content. The system then applies a dual preprocessing pipeline consisting of two complementary tracks. The first, known as the semantic track, extracts and preserves phishing-related indicators without altering the original content, generating a rich set of metadata features such as suspicious URLs, hidden links, phishing keywords, authentication failures (SPF, DKIM, DMARC), domain mismatches, and behavioral signals like urgency or fear. The second, the structural track, cleans and normalizes the email body by removing HTML tags, decoding entities, and replacing elements such as URLs, email addresses, and monetary values with standardized tokens, producing text suitable for machine learning models. These outputs are then passed to two separate classifiers: a header analysis model that evaluates the extracted metadata and a body analysis model that processes the cleaned text alongside semantic features. Each model returns a risk probability, and the system computes the final decision based on the highest score compared to a predefined threshold. If the risk exceeds the threshold, the email is dropped; otherwise, it is forwarded to the internal mail server using SMTP. Throughout this process, detailed logs are generated to record message metadata, model outputs, and system decisions, supporting monitoring and evaluation. By combining asynchronous SMTP handling, modular model integration, and a two-layer analysis approach, the script provides an efficient and scalable solution for real-time detection and filtering of phishing and malicious emails.

In every time we need to start the service, we will need to start it manually by using `python3 mail_filter.py`,so we Configuring `mail_filter.py` as a system service (e.g., using systemd) to ensure that this filter is always running in the

background, starts automatically on boot, and restarts if it crashes. Without running it as a service, Postfix would try to send emails to port 10025 and fail if the script is not manually started, breaking the mail flow.

On fig. 6.4.4, we create `/etc/systemd/system/mail-filter.service` a file and add the configuration, making `mail_filter.py` an always-running SMTP service that Postfix depends on.

```
  GNU nano 4.8          /etc/systemd/system/mail-filter.service
[Unit]
Description=MGW Mail Filter - SMTP Proxy with ML/NLP Analysis
After=network.target postfix.service
Wants=postfix.service

[Service]
Type=simple
User=rama
Group=rama
WorkingDirectory=/home/rama/MGW
Environment=PATH=/home/rama/MGW/mail-env/bin:/usr/local/bin:/usr/bin:/bin
ExecStart=/home/rama/MGW/mail-env/bin/python3 /home/rama/MGW/mail_filter.py
Restart=always
RestartSec=5
StandardOutput=append:/var/log/mail_filter.log
StandardError=append:/var/log/mail_filter.log

[Install]
WantedBy=multi-user.target
```

Fig. 6.4.4: configure mail_filter.py system service

The `mail-filter.service` ensures that the Python script (`mail_filter.py`) is continuously running as a background process, listening on port 10025 as fig. 6.4.5, which is exactly the endpoint defined in `main.cf` using `content_filter = smtp:[127.0.0.1]:10025`. This creates a direct link: Postfix receives the email and forwards it to port 10025, and the Python filter (kept alive by systemd) receives and processes it. The `After=network.target postfix.service` and `Wants=postfix.service` ensure proper startup order so the filter runs alongside Postfix, while `Restart=always` guaranteeing high availability by automatically restarting the filter if it crashes. Additionally, the virtual environment setup (`mail-env`) ensures all ML/NLP dependencies are available, and logging `/var/log/mail_filter.log` helps monitor filtering activity and debug issues.

In short, Postfix defines the flow, while systemd ensures the filter is alive to handle that flow, and `mail_filter.py` performs the actual processing—together forming a reliable, production-style mail gateway pipeline.

```
rama@mgw:~$ sudo systemctl status mail-filter
● mail-filter.service - MGW Mail Filter - SMTP Proxy with ML/NLP Analysis
     Loaded: loaded (/etc/systemd/system/mail-filter.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2026-04-14 22:57:40 IDT; 1h 41min ago
   Main PID: 4053 (python3)
      Tasks: 4 (limit: 8204)
     Memory: 786.4M
     CGroup: /system.slice/mail-filter.service
             └─4053 /home/rama/MGW/mail-env/bin/python3 /home/rama/MGW/mail_filter.py

Apr 14 22:57:40 mgw.company.com systemd[1]: mail-filter.service: Succeeded.
Apr 14 22:57:40 mgw.company.com systemd[1]: Stopped MGW Mail Filter - SMTP Proxy with ML/NLP Analysis.
Apr 14 22:57:40 mgw.company.com systemd[1]: Started MGW Mail Filter - SMTP Proxy with ML/NLP Analysis.
```

Fig. 6.4.5: mail-filter system service

6.3.2 Email Feature Extraction Pipeline for Phishing Detection "Parsing"

Email parsing with datasets extracts and analyzes different components of incoming messages such as headers, body content, links, and attachments so that they can be evaluated against known patterns of spam, phishing, or malicious behavior. By using datasets of emails (e.g., labeled spam vs. legitimate messages), the system can apply rule-based filtering or machine learning techniques to improve detection accuracy, reduce false positives, and adapt to new threats. This process is critical for identifying suspicious characteristics, enforcing security policies, and ensuring that only safe and legitimate emails are forwarded to the internal mail server.

In Fig. 6.4.6, we can see the content of the email feature extraction mode files and script list.

```
rama@mgw:~/MGW/Parsing$ ls -l
total 43688
-rw-rw-r-- 1 rama rama     7186 Mar 26 18:15 enron_features_feature_descriptions.json
-rw-rw-r-- 1 rama rama 13809341 Mar 26 18:15 enron_features_features.csv
-rw-rw-r-- 1 rama rama    37359 Mar 26 18:15 extract_phishing_features.py
-rw-rw-r-- 1 rama rama     7186 Mar 26 18:15 phishing_features_feature_descriptions.json
-rw-rw-r-- 1 rama rama 30864463 Mar 26 18:15 phishing_features_features.csv
```

Fig. 6.4.6: Email Feature Extraction Pipeline "Parsing model" content

The provided Python script `"extract_phishing_features.py"` represents a complete system for analyzing suspicious emails within the MGW project. Its core idea is based on splitting the analysis into two parallel tracks. The first track focuses on analyzing the email content and metadata without executing anything potentially harmful, while the second track focuses on extracting files and artifacts from the email and preparing them for sandbox-based dynamic analysis. This design is important because it combines fast detection (static analysis) with deeper inspection (dynamic analysis), which is exactly how modern enterprise email security gateways operate.

When an email enters the system, it is first parsed into a structured object using Python's email library. After that, the processing is divided into Track A and Track B. In Track A, the system extracts the email content, whether it is HTML or plain text. If plain text is not available, the HTML is converted into readable text. The content is then cleaned using BeautifulSoup, where non-essential elements such as scripts and styles are removed while preserving meaningful textual data. A key idea here is not just cleaning the text, but preparing it for machine learning. Instead of removing variable elements like URLs, email addresses, or IPs, they are replaced with standardized tokens such as URL_TOKEN and EMAIL_TOKEN. This allows the model to learn patterns rather than memorizing specific values.

Once the text is prepared, the system extracts a rich set of features. One of the most critical components is URL analysis. The system counts the number of URLs, detects URLs that use IP addresses instead of domain names, identifies mismatches between displayed links and actual links (a common phishing technique), and checks for suspicious top-level domains. It also evaluates whether links use HTTP or HTTPS and measures their lengths. This stage provides some of the strongest indicators for phishing detection. In addition, the system analyzes keyword patterns associated with social engineering, such as urgency, fear, and curiosity. These are converted into numerical scores that reflect the psychological tactics used by attackers.

The code also computes detailed text statistics, including uppercase ratios, punctuation usage, word counts, and unique word distributions. These features help machine learning models distinguish between legitimate and malicious messages. Furthermore, the HTML structure itself is analyzed to detect potentially dangerous elements such as embedded JavaScript, forms requesting sensitive input, base64-encoded content, and data URIs. These techniques are commonly used in modern phishing attacks to hide malicious behavior.

Another critical part of Track A is header analysis. The system inspects email authentication mechanisms such as SPF, DKIM, and DMARC to determine if they failed. It also checks for inconsistencies between the sender domain, reply-to, and return-path fields to detect spoofing attempts. Additionally, the subject line is analyzed for suspicious patterns such as urgency keywords, excessive capitalization, special characters, or financial terms. These features are essential in identifying phishing attempts at an early stage.

Track B operates in a completely different way by focusing on extracting actual artifacts from the email. Attachments are extracted and stored in a dedicated directory called sandbox_dropzone. Filenames are sanitized to prevent exploitation techniques such as path traversal. Each file is saved and hashed using MD5 and SHA256, and then classified based on its type, such as executable files, archives, documents, or images. The presence of certain types, especially executables, is considered a strong risk indicator.

In addition to attachments, the system extracts any embedded JavaScript code from the email and saves it as separate files. It also detects and decodes data

embedded using data URIs and converts them into real files. This is particularly important because attackers often hide malicious payloads inside HTML content. After extracting all artifacts, the system generates a JSON manifest file that contains detailed information about each extracted object, including file paths and hashes. This manifest serves as a bridge to external analysis tools such as Cuckoo Sandbox or VirusTotal.

Finally, the outputs of both tracks are combined into a single result. The system returns a comprehensive feature set that includes text-based features, statistical analysis, URL and header insights, as well as artifact-related information such as attachment counts and the path to the sandbox manifest. This combined output can be directly used by a machine learning model for classification, while also enabling deeper investigation through sandbox analysis when needed.

Overall, this design is powerful because it separates fast static analysis from deeper dynamic preparation, allowing the system to handle real-time email filtering while still being capable of analyzing advanced threats. It is also highly scalable and modular, making it easy to integrate machine learning models or external security tools without changing the core architecture.

```
python3 /home/rama/MGW/Parsing/extract_phishing_features.py
~/Datasets/master_phishing_dataset.csv
```

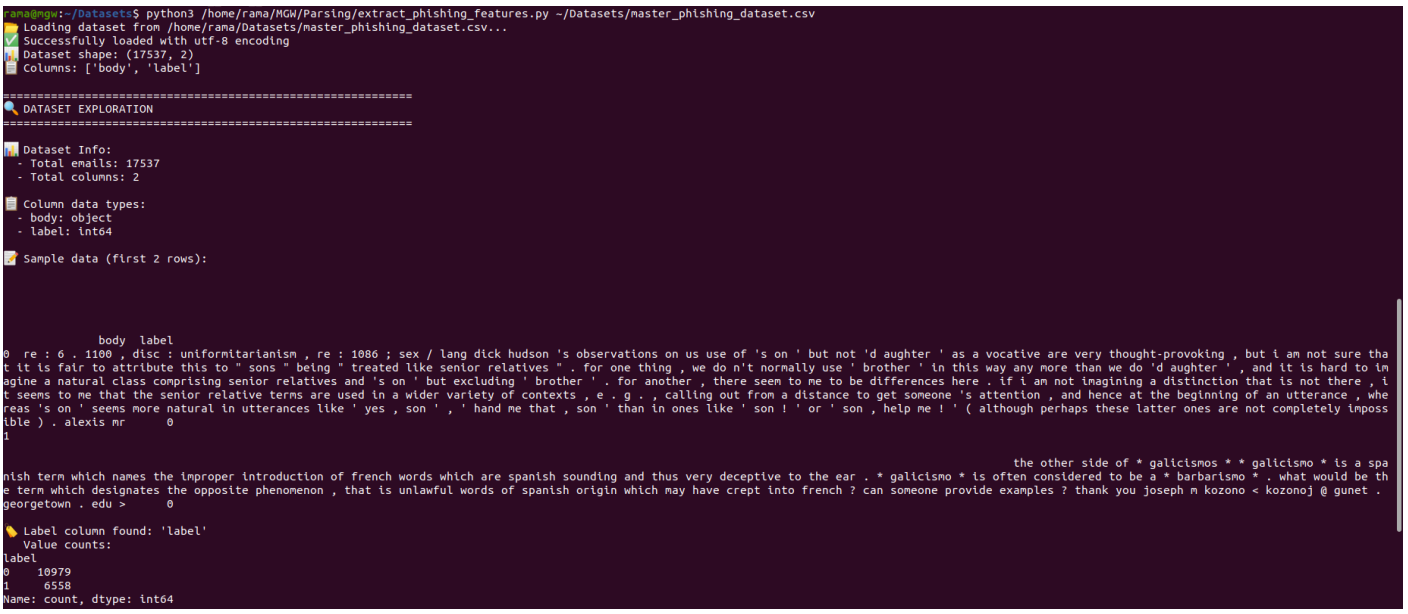


Fig. 6.4.7a: dataset statistic information

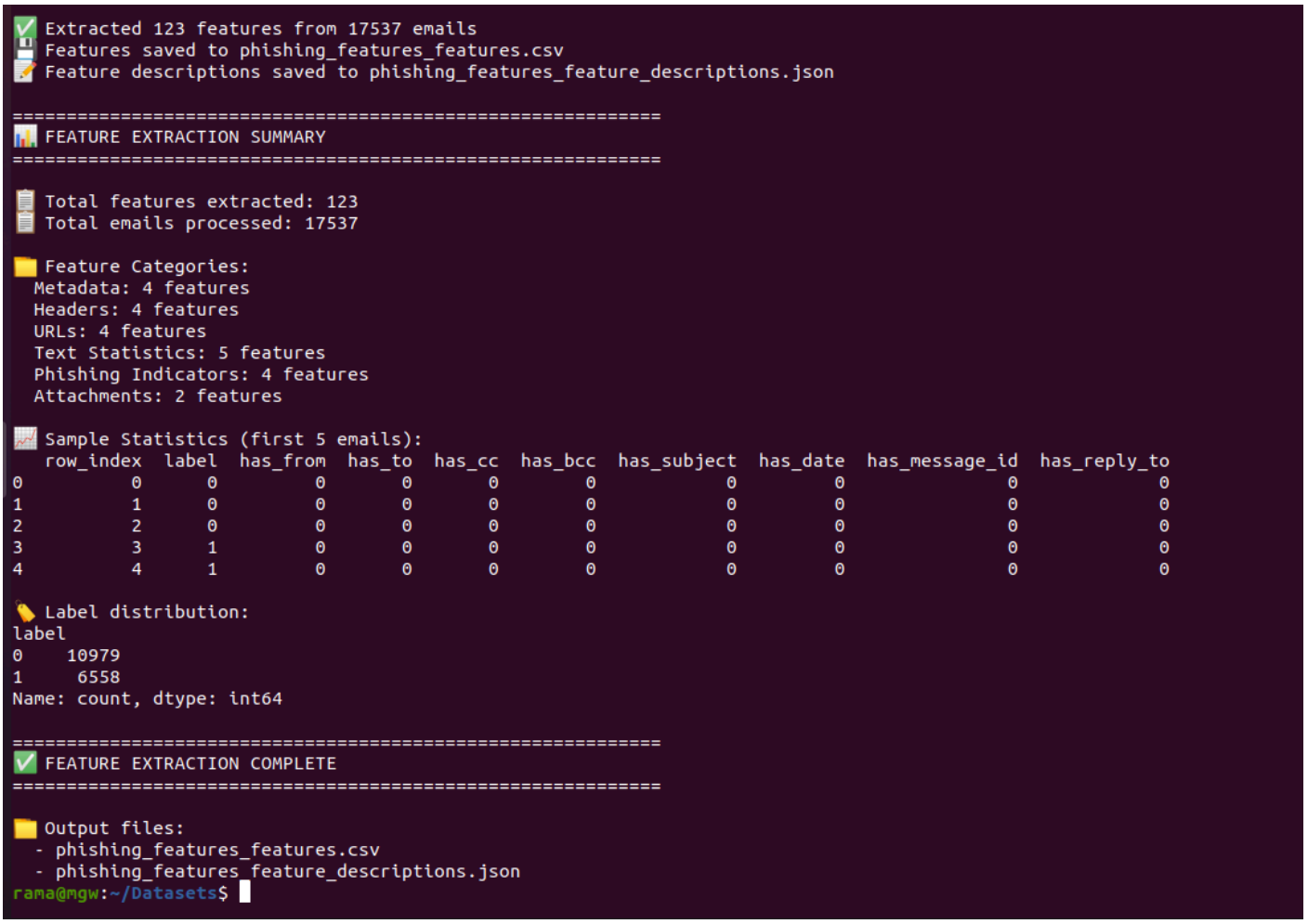


fig 6.4.7b: statistic information for features extracted

The fig. 6.4.6 shows the main functionality lies in the feature extraction process, where each email is parsed into multiple categories of features:

- **Header Features:** Extracts metadata such as sender, recipients, subject characteristics, and timestamps. It identifies patterns like suspicious subject keywords (e.g., "urgent" and “verify”), capitalization ratios, and sender domains.
- **Body Features:** Performs deep text analysis, including length, word count, character distribution, and linguistic patterns. It also detects phishing indicators such as excessive punctuation, abnormal capitalization, and keyword frequency.
- **URL Analysis:** Extracts and analyzes all links within the email, checking for suspicious properties such as IP-based URLs, unusual top-level domains, excessive subdomains, or encoded characters—common traits in phishing attacks.
- **Security Indicators:** Identifies elements like embedded email addresses, phone numbers, IP addresses, JavaScript code, HTML forms, and encoded content, which may indicate malicious intent.
- **Behavioral Features:** Calculates scores based on urgency, fear, and curiosity triggers, which are commonly used in social engineering attacks.
- **Attachment Features:** Detects the presence and type of attachments (e.g., executable, archive, document), which are often used to deliver malware.

Each processed email is converted into a structured feature vector, and all features are aggregated into a new dataset (`features_df`). This dataset is then exported as a CSV file, while a corresponding JSON file provides detailed descriptions of each feature for interpretability and documentation purposes.

Overall, the system achieves the transformation of unstructured email data into a rich, high-dimensional feature set. This enables advanced email filtering in a secure gateway environment, where these features can be integrated into detection mechanisms such as rule-based engines or machine learning classifiers to accurately identify phishing and malicious emails.

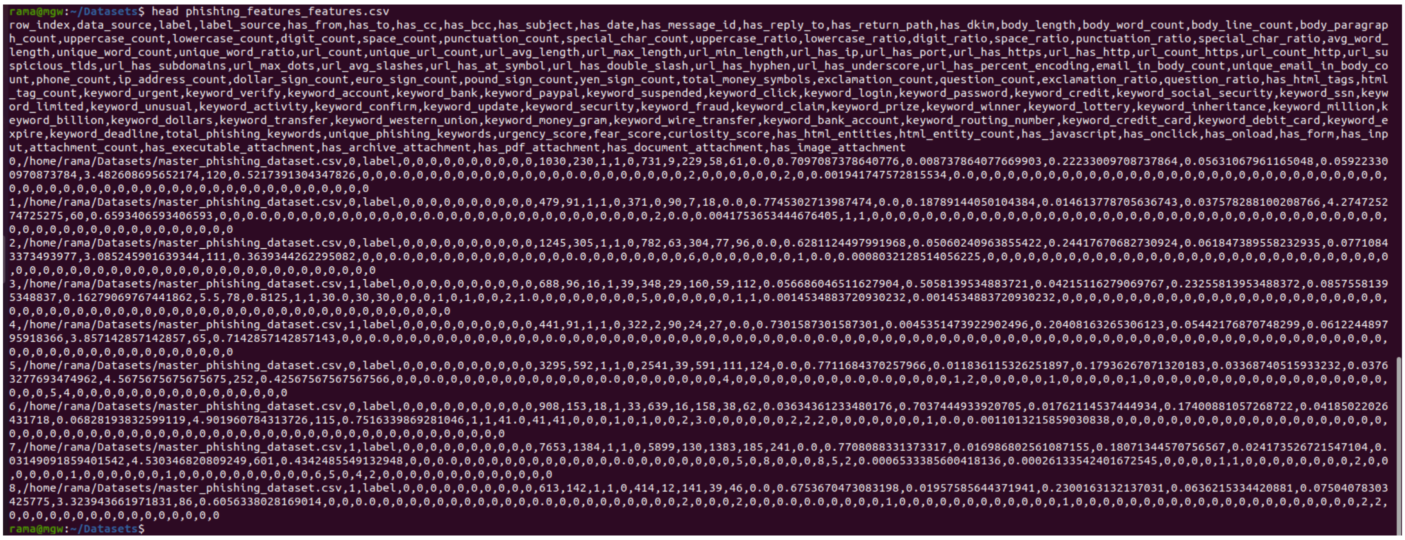


Fig. 6.4.8: Extracted features

6.3.3 Header model

The `header.py` script processes data by receiving a structured set of features that were previously extracted from the email during the semantic preprocessing stage in the main filter. These features represent measurable indicators from the email header, such as authentication results (e.g., SPF, DKIM, DMARC), domain relationships, subject characteristics, and URL-related signals. The script does not extract raw data itself; instead, it operates on this already-prepared feature vector, ensuring a clear separation between feature extraction and classification.

Once the feature data is received, the script begins by determining how the analysis will be performed. It first attempts to load a previously trained XGBoost model from disk. If the model is available, it is used directly to process the input data. If not, the script attempts to train a new model using available labeled datasets. This training process involves selecting relevant feature columns, converting them into numerical format, splitting the dataset into training and testing subsets, and fitting the XGBoost classifier to learn patterns that distinguish phishing emails from legitimate ones. If neither a trained model nor a suitable dataset is available, the system falls back to a heuristic approach, ensuring that the analysis can still proceed.

In the model-based approach, the input features are arranged into a single-row data structure compatible with the trained model. The XGBoost classifier then processes this data by passing it through an ensemble of decision trees. Each tree contributes to the final prediction by evaluating different combinations of features, and their outputs are aggregated to produce a probability score. This probability represents how likely the email header is to be associated with a malicious or phishing message. The final risk probability is obtained using the `predict_proba` function, which outputs a value between 0 and 1, where values closer to 1 indicate higher risk.

In the heuristic mode, the risk evaluation is computed differently. Each feature is multiplied by a predefined weight that reflects its importance or risk contribution. Positive weights increase the likelihood of the email being malicious, while negative weights reduce it, representing legitimate characteristics. These contributions are summed to form a net score, which is then passed through a mathematical normalization function (based on the hyperbolic tangent) to map the result into a probability value between 0 and 1. This ensures consistency with the model-based output format.

Finally, the script produces the risk evaluation result, which includes the computed probability, a list of the most influential features (either based on model feature importance or heuristic contributions), and metadata such as the timestamp and the analysis method used. This probability serves as a key input for the main filtering system, which uses it to decide whether to allow or block the email.

The fig. 6.4.9, shown in the directory, represents the **header analysis module** of the Secure Email Gateway system, containing the essential components required to evaluate email header features. The main script, `header.py`, implements the logic for analyzing structured metadata extracted from email headers and computing a risk probability using either a machine learning model (XGBoost) or a fallback heuristic method. The file `header.log` is used to record detailed information about the analysis process, including computed risk scores, selected features, and any errors encountered during execution, which supports system monitoring and debugging.

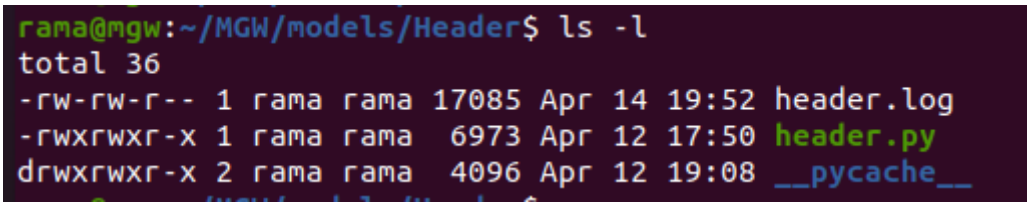


Fig. 6.4.9: The Header directory model

The `__pycache__` directory contains automatically generated compiled Python files that improve execution performance by reducing the need to recompile the script on each run. Although it does not directly contribute to the analysis logic, it enhances the efficiency of the module. Notably, unlike the body analysis module, no pre-trained model file is currently present in this directory, which indicates

that the system may either load the model from another location, train it dynamically at runtime using available datasets, or rely on the heuristic approach when a trained model is not available.

Overall, this directory provides a modular and flexible environment for header-based risk evaluation, enabling the system to analyze authentication results, structural properties, and metadata signals of emails to support accurate phishing detection.

In figures 6.4.10 - 6.4.11, we show two different emails with different levels of risk probability, and the shown figures show how the model evaluates the risk probability.

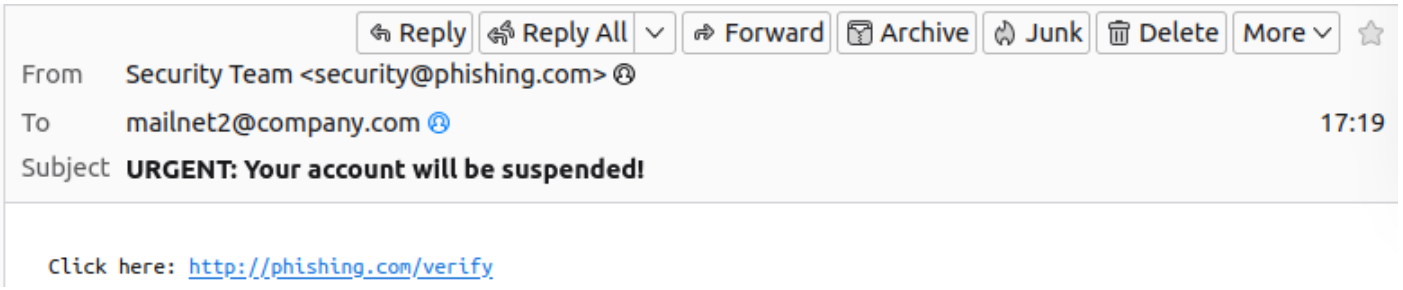


Fig. 6.4.10a: mail 1 content and header

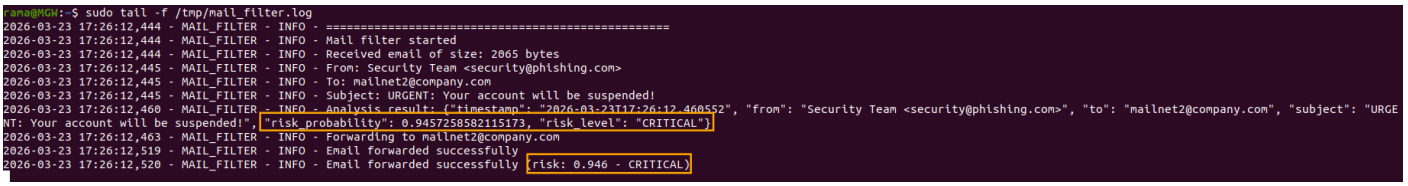


Fig. 6.4.10b: header analysis for mail 1

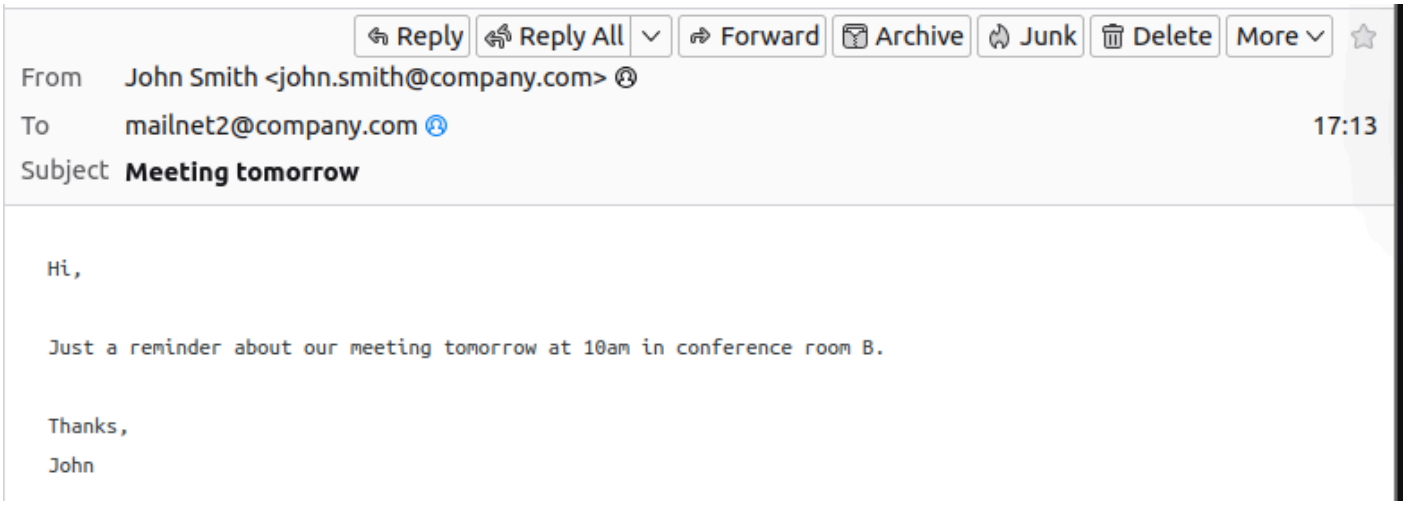


Fig. 6.4.11a: mail 2 content & header

Fig. 6.4.11b: Header analysis for mail 2

6.3.4 BERT-Based Email Body Analysis Module

The `body.py` script is responsible for analyzing the **content of the email body** and estimating its risk level using a combination of deep learning, classical machine learning, and semantic feature scoring. It processes two main inputs: the cleaned email text produced by the structural preprocessing stage and the semantic metadata extracted earlier. The goal of this component is to evaluate how suspicious the actual message content is and produce a probability indicating whether it is phishing or malicious.

The processing begins by attempting to use a **deep learning model based on RoBERTa**, which is a transformer-based language model capable of understanding the contextual meaning of text. If a fine-tuned version of the model already exists locally, it is loaded directly. Otherwise, the script loads a base pre-trained model and attempts to fine-tune it using a labeled phishing dataset if available. During fine-tuning, the model learns patterns from email text by tokenizing the input, converting it into numerical representations, and training on labeled examples to distinguish between legitimate and phishing messages. When analyzing a new email, the cleaned text is tokenized and passed through the model, which outputs logits (raw predictions). These are then converted into probabilities using a softmax function, producing a value between 0 and 1 that represents the likelihood of the email being malicious. This value is referred to as the **NLP-based probability**.

If RoBERTa cannot be used (for example, due to missing resources or errors), the system falls back to a **TF-IDF + Logistic Regression model**. In this approach, the text is converted into numerical feature vectors based on word frequency and importance (TF-IDF) and then classified using logistic regression. This provides a simpler but effective alternative for estimating the risk probability from the text.

In parallel with the NLP analysis, the script computes a **semantic risk score** using predefined weights applied to the metadata features. Each feature (such as suspicious URLs, embedded scripts, phishing keywords, urgency indicators, or authentication failures) contributes positively or negatively to the risk score depending on its weight. These contributions are summed to produce a net score, which is then normalized using a mathematical function (tanh) to produce a probability value between 0 and 1. This represents how risky the email appears

based on its structural and behavioral characteristics rather than its textual meaning.

The final risk probability is computed using a **fusion mechanism** that combines both sources of information. If an NLP-based probability is available, the system calculates a weighted average where **55% is assigned to the NLP model (text understanding)** and **45% to the semantic metadata score (behavioral indicators)**. This ensures that both the meaning of the text and the presence of technical phishing signals are considered. If NLP analysis is not available, the system relies entirely on the semantic score.

Finally, the script outputs a structured result that includes the final risk probability, the most important contributing factors, the timestamp, and the analysis method used (RoBERTa, TF-IDF, or semantic-only). This probability is then used by the main filtering system to determine whether the email should be allowed or blocked.

The fig 6.4.12, shown the directory shown represents the **body analysis module environment** of the Secure Email Gateway system, containing all components required for processing and classifying email content. The main script, `body.py`, implements the logic for analyzing email body text and computing its risk probability using multiple techniques. The file `body.log` stores detailed runtime logs, including model outputs, errors, and processing information, which supports monitoring and debugging. The `roberta_finetuned` directory contains the fine-tuned version of the RoBERTa model, which has been trained on phishing datasets to improve its ability to understand and classify email text based on context and language patterns.

```
rama@mgw:~/MGW/models/Body$ ls -l
total 75680
-rw-rw-r-- 1 rama rama    11155 Apr 14 19:52 body.log
-rwxrwxr-x 1 rama rama    10238 Apr 12 17:48 body.py
drwx------ 2 rama rama     4096 Apr 12 19:08 __pycache__
drwxr-xr-x 2 rama rama     4096 Apr 12 19:22 roberta_finetuned
-rw-r--r-- 1 rama rama 76734449 Apr 12 00:01 tfidf_body_model.pkl
-rw-r--r-- 1 rama rama   722440 Apr 12 19:36 tfidf_fallback.pkl
```

Fig. 6.4.12: The model directory model

In addition, the system includes two serialized machine learning models stored as `.pkl` files. The file `tfidf_body_model.pkl` represents a larger TF-IDF-based model that may have been trained on a more extensive dataset, while `tfidf_fallback.pkl` is a lighter fallback model used when the primary deep learning model is unavailable or fails during execution. These models enable the system to maintain functionality under different conditions, ensuring robustness.

The `__pycache__` directory contains compiled Python bytecode files that improve execution performance but do not affect the system logic directly.

Together, these components form a complete and modular environment for body content analysis, where deep learning and traditional machine learning models coexist with logging and caching mechanisms to provide efficient, scalable, and reliable phishing detection.

The figs 6.4.13, shown in the first stage, show an attacker manually sending two phishing emails using a raw SMTP session via `telnet`. The attacker connects to the mail gateway on port 25 and manually issues SMTP commands such as `HELO`, `MAIL FROM`, `RCPT TO`, and `DATA`. This simulates a realistic phishing attempt entering the mail system.

Once an email is received, it is analyzed by the Mail Gateway Filter (`mail_filter.py`). In fig. 6.4.14, The header analysis reveals a risk score of 0.58, indicating moderate suspicion due to weak sender indicators and lack of authentication. The body analysis yields a higher risk score of 0.83, suggesting a strong phishing likelihood based on suspicious TLDs, manipulative language, and phishing keywords. The system employs a TF-IDF + semantic fusion model due to RoBERTa tokenizer failure. The final risk score of 0.8317 exceeds the set threshold of 0.70, resulting in the email being classified as malicious and blocked from delivery.

```
rama@rama:~$ telnet mgw.company.com 25
Trying 192.168.200.254...
Connected to mgw.company.com.
Escape character is '^]'.
220 mgw.company.com ESMTP Postfix (Ubuntu)
HELO mail.newsletter.com
MAIL FROM: <offers@daily-deals.com>
RCPT TO: <target@example.com>
DATA
From: "Daily Deals" <offers@daily-deals.com>
Subject: You won!
X-Forwarded-For: 8.8.8.8

CONGRATULATIONS!!!
Click here to claim your FREE iPhone:
http://fake-winner.xyz/claim

Limited time only.
Act now.
.
QUIT250 mgw.company.com
250 2.1.0 Ok
250 2.1.5 Ok
354 End data with <CR><LF>.<CR><LF>
250 2.0.0 Ok: queued as 2F942160D4B

221 2.0.0 Bye
Connection closed by foreign host.
rama@rama:~$
```

Fig. 6.4.13: Attacker Email Injection (SMTP Simulation)

```
2026-04-14 23:01:06,876 [WARNING] Only 0 header cols in dataset — heuristic only
2026-04-14 23:01:06,913 [INFO] {"risk_probability": 0.581759, "risk_factors": ["subject_caps_ratio=0.12 w=+0.30", "sub
ject_exclamation=1.00 w=+0.10"], "timestamp": "2026-04-14T20:01:06.913599", "engine": "heuristic"}
2026-04-14 23:01:06,914 [INFO] Loading fine-tuned RoBERTa
2026-04-14 23:01:06,917 [WARNING] RoBERTa failed: Couldn't instantiate the backend tokenizer from one of:
(1) a `tokenizers` library serialization file,
(2) a slow tokenizer instance to convert or
(3) an equivalent slow tokenizer class to instantiate and convert.
You need to have sentencepiece or tiktoken installed to convert a slow tokenizer to a fast one. — trying TF-IDF
2026-04-14 23:01:06,943 [INFO] {"risk_probability": 0.831652, "risk_factors": ["nlp_score=0.9127", "semantic_score=0.7
326", "url_suspicious_tlds=1.00 w=+0.30", "urgency_score=0.20 w=+0.30", "curiosity_score=0.80 w=+0.20", "total_phishin
g_keywords=5.00 w=+0.04"], "timestamp": "2026-04-14T20:01:06.943742", "engine": "tfidf+semantic"}
2026-04-14 23:01:06,944 [INFO]
---------NEW Email message-----------------------
[2026-04-14 20:01:06 UTC] <gen-1776196860.2133234@mgw>
from: "Daily Deals" <offers@daily-deals.com> -> to: unknown
Header Analyzer = [0.5818]
Body Analyzer   = [0.8317]
------Email Analyzer DONE-----------------------
2026-04-14 23:01:06,944 [WARNING] [DROPPED] <gen-1776196860.2133234@mgw> risk=0.8317 >= 0.7
```

Fig. 6.4.14: Mail Gateway Processing and Security Analysis

In fig. 6.4.15, shown Postfix logs detail the system's interaction with the mail infrastructure, indicating that emails are first accepted and then forwarded to a local filtering gateway. Safe emails are queued and delivered, while malicious ones are dropped. Specifically, the message with ID <`2F942160D4B`> was rejected. The logs also highlight SMTP protocol anomalies, such as improper command pipelining, which mimic potential attacks, underscoring the gateway's robustness.

```
Apr 14 23:00:54 mgw postfix/smtpd[4066]: connect from unknown[192.168.200.100]
Apr 14 23:01:00 mgw postfix/smtpd[4066]: improper command pipelining after MAIL from unknown[192.168.200.100]: RCPT TO
: <target@example.com>\r\nDATA\r\nFrom: "Daily Deals" <offers@daily-deals.com>\r\nSubject: You won!
Apr 14 23:01:00 mgw postfix/smtpd[4066]: 2F942160D4B: client=unknown[192.168.200.100]
Apr 14 23:01:00 mgw postfix/cleanup[4069]: 2F942160D4B: message-id=<>
Apr 14 23:01:00 mgw postfix/qmgr[1182]: 2F942160D4B: from=<offers@daily-deals.com>, size=400, nrcpt=1 (queue active)
Apr 14 23:01:00 mgw postfix/smtp[4070]: 2F942160D4B: to=<target@example.com>, relay=127.0.0.1[127.0.0.1]:10025, delay=
0.03, delays=0.02/0.01/0/0, dsn=2.0.0, status=sent (250 OK: queued)
Apr 14 23:01:00 mgw postfix/qmgr[1182]: 2F942160D4B: removed
Apr 14 23:01:01 mgw postfix/smtpd[4066]: disconnect from unknown[192.168.200.100] helo=1 mail=1 rcpt=1 data=1 quit=1 c
ommands=5
```

Fig. 6.4.15: Mail Transfer System Behavior (Postfix Logs)

6.3.5 Attachment model

In this section, we introduce the dynamic analysis layer of the Mail Gateway (MGW). To protect the internal network from advanced persistent threats (APTs) and zero-day malware embedded in email attachments, a robust, automated sandboxing solution is required. For this purpose, CAPEv2 (Configurable Analysis and Protection Engine) was selected as the core malware analysis platform.

What is CAPEv2?

CAPEv2 is an open-source, highly sophisticated automated malware analysis system derived from Cuckoo Sandbox. It is specifically designed to execute suspicious files in isolated environments and monitor their behavior in real-time.

Key Features and Advantages of CAPEv2:

- Malware Unpacking and Payload Extraction: Unlike standard sandboxes, CAPEv2 excels at advanced memory inspection, allowing it to automatically unpack malware and extract configurations/payloads directly from memory.
- Extensive API Support: It provides a powerful Web API that allows seamless integration with our core Mail Gateway components.
- Rich Behavior Logging: It captures API calls, network traffic (PCAP), file system modifications, and registry changes during execution.

2. Proposed Attachment Processing Logic

The integration between the Mail Gateway and the sandboxing cluster follows a strict, isolated pipeline to ensure high throughput and maximum security. The operational workflow is structured as follows:

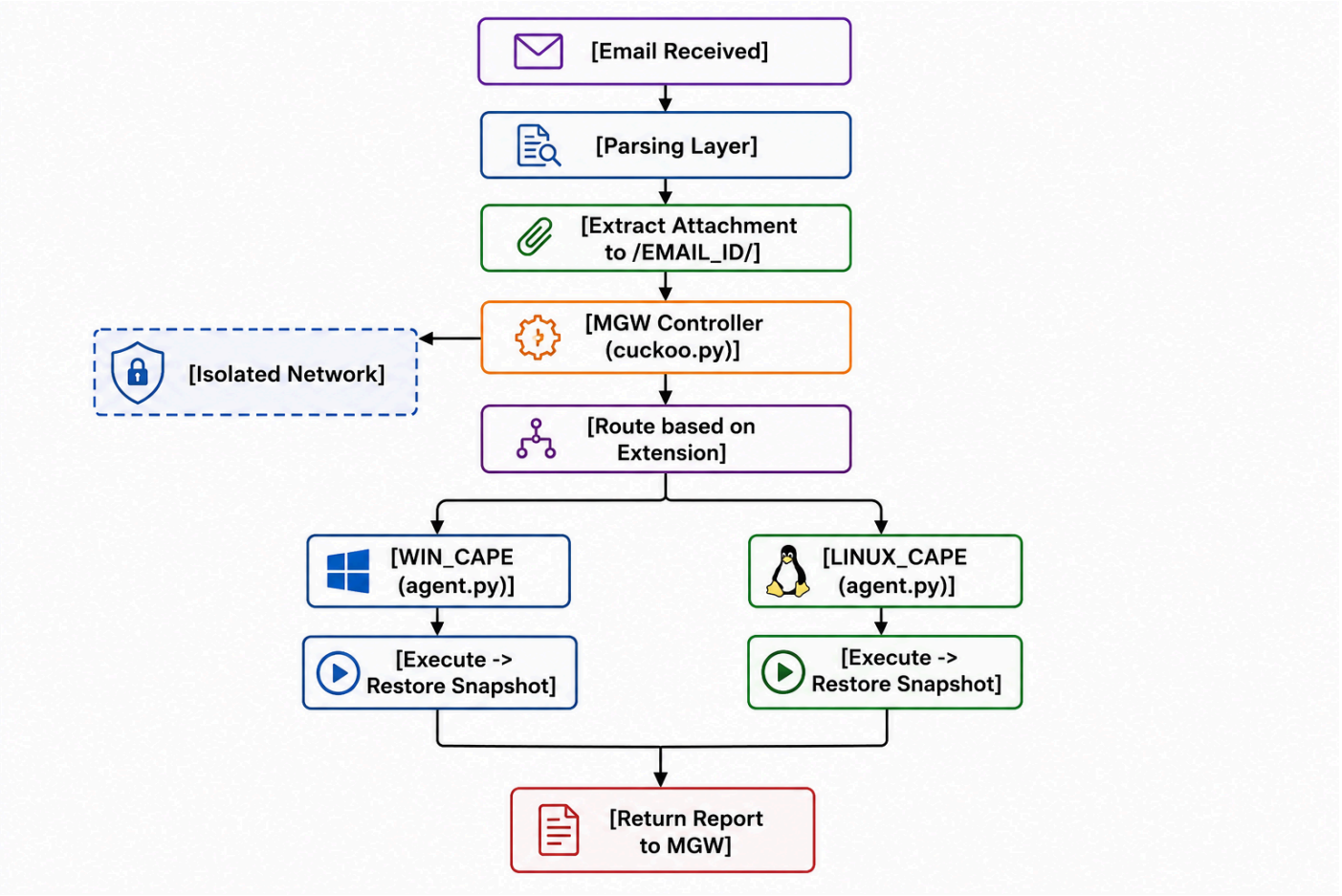


Fig. 6.4.16: Attachment processing logic

A. Extraction and Parsing Phase

1. Parsing & Tracking: Once an email is intercepted by the gateway, the parsing layer scans the message components.
2. Directory Isolation: If an attachment is detected, a tracking mechanism extracts the file and saves it into a dedicated directory named dynamically based on the unique EMAIL_ID (e.g., /attachments/EMAIL_ID/). This ensures strict data segregation and alignment with the specific email session.

B. Dynamic Routing and Dynamic Analysis

3. Extension-Based Routing: The MGW controller analyzes the file extension. Files are conditionally routed to the appropriate analysis environment:
    - Windows-specific files (e.g., .exe, .docx, .xlsx) are routed to the Windows Sandbox.
    - Linux-specific files (e.g., .sh, .elf) are routed to the Linux Sandbox.

- Cross-Platform / Shared Extensions (e.g., .pdf, .zip containing mixed files) are duplicated and routed to both environments simultaneously to observe behaviors across different operating systems.

C. Execution and VM Restoration

4. Isolated Network Transport: The files are pushed from the MGW to the respective virtual machines over a strictly isolated network segment, preventing any potential malware from escaping into the production environment.
5. Agent Execution & Snapshot Reversion: Inside each guest environment, a lightweight agent executes the sample. Once the analysis window closes:
   - The sandbox automatically discards all modifications by restoring the clean snapshot state of the VM.
   - This guarantees that the next attachment is analyzed in a completely untainted, fresh environment.
6. Result Aggregation: The final analysis results and threat scores are compiled and securely transmitted back to the Mail Gateway to determine whether the email should be delivered or quarantined.

3. Architectural Component Mapping

To clarify the deployment structure of our architecture, the system components are mapped as follows:

Component Role	System / Script Designation	Description
Core Master (MGW Side)	CAPEv2 Full Stack, cuckoo.py, Report Generator	Acts as the central brain. It handles the API requests, manages the submission queues, coordinates file routing based on extensions, processes the final JSON/HTML reports, and decides the fate of the email.
Analysis Nodes (Guests)	WIN_CAPE & LINUX_CAPE (agent.py)	Isolated virtual machine instances (Windows and Linux) running the CAPEv2 Python agent.py. These nodes are responsible solely for executing the attachment, recording behavioral data, and interacting with the Master host.

3. Implementation Process

1. By fig.17a-e, we Setup an static ip for the MGW and both sandbox

```
rama@mgw:~/Desktop$ sudo cat /etc/netplan/01-network-manager-all.yaml
[sudo] password for rama:
# Let NetworkManager manage all devices on this system
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.200.254/24
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
      routes:
        - to: default
          via: 192.168.200.1
    enp0s8:
      dhcp4: no
      addresses:
        - 10.10.0.10/24
    enp0s9:
      dhcp4: no
      addresses:
        - 10.0.0.10/24
rama@mgw:~/Desktop$
```

Fig. 6.4.17.a: the mgw network file instruction

```
rama@mgw:~/Desktop$ ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:29:94:d9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.254/24 brd 192.168.200.255 scope global enp0s3
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe29:94d9/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:cf:e0:65 brd ff:ff:ff:ff:ff:ff
    inet 10.10.0.10/24 brd 10.10.0.255 scope global enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fecf:e065/64 scope link
       valid_lft forever preferred_lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e7:e3:c8 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.10/24 brd 10.0.0.255 scope global enp0s9
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fee7:e3c8/64 scope link
       valid_lft forever preferred_lft forever
rama@mgw:~/Desktop$
```

Fig. 6.4.17.b: The MGW machine ip's

Open

01-network-manager-all.yaml
/etc/netplan

```
network:
  version: 2
  ethernets:
    enp0s3:
      addresses: [10.0.0.20/24]
      routes:
        - to: default
          via: 10.0.0.10
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]
```

Fig. 6.4.17.c: Linux sandbox network file formate

```
rama@CAPE:~/Desktop$ ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
 1000
    link/ether 08:00:27:86:7f:55 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.20/24 brd 10.0.0.255 scope global enp0s3
       valid_lft forever preferred_lft forever
rama@CAPE:~/Desktop$
```

Fig. 6.4.17.d: Linux sandbox network file formate

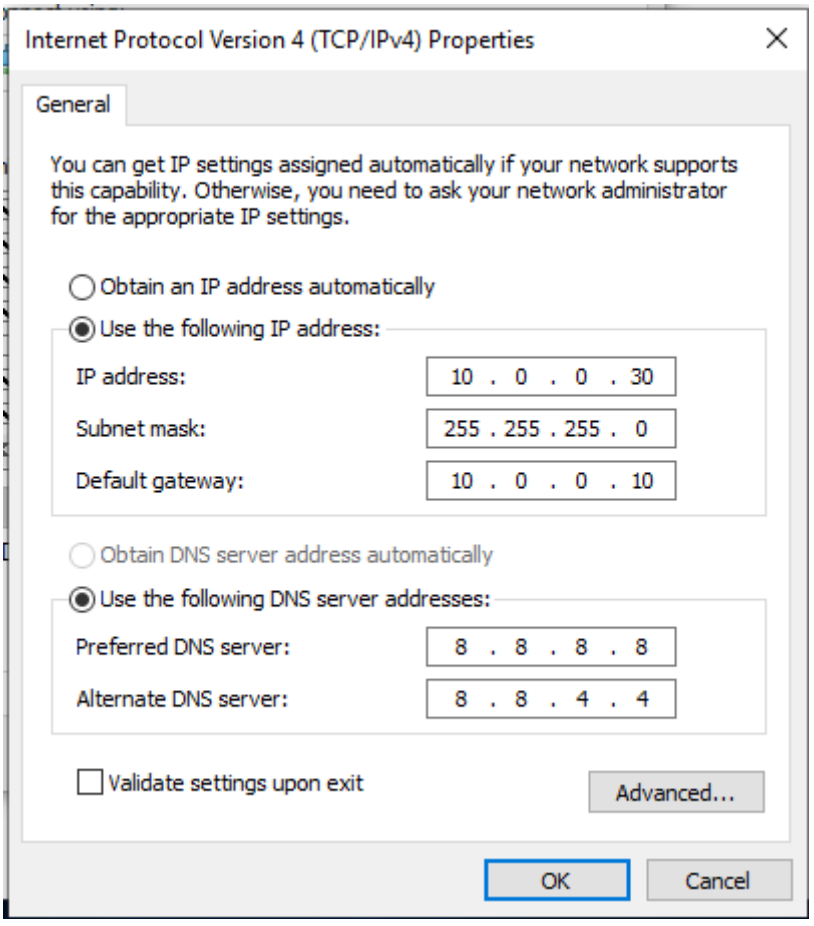


Fig. 6.4.17.e: windows sandbox IPv4 settings

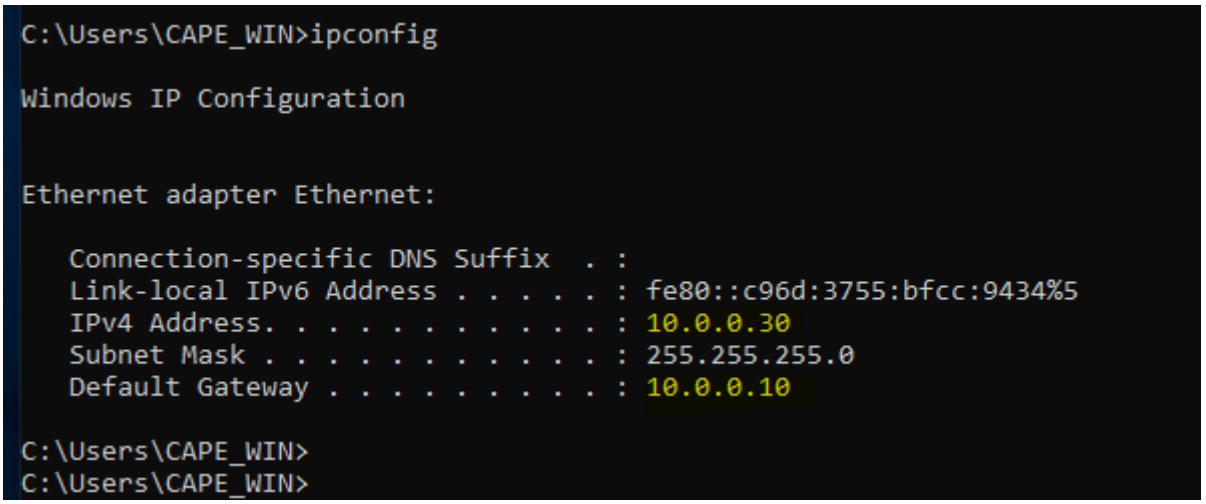


Fig. 6.4.17.f : windows sandbox IPv4

Now, we need to ensure that both sandbox's can successfully connect with the MGW machine, to can deliver to them the attachment for analysis by testing Connectivity, from all machines through figs (6.4.18a-c)

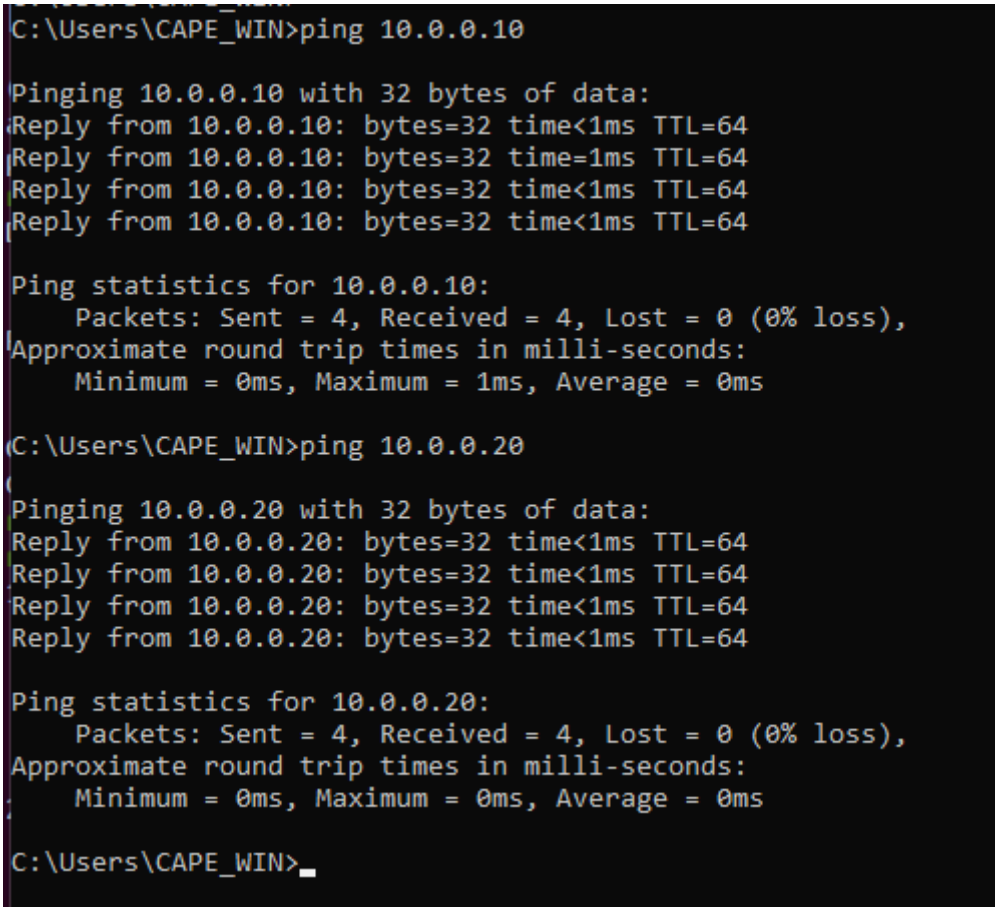


Fig. 6.4.18.a: test connectivity from the win sandbox, for the linux Sandbox & MGW

```
rama@CAPE:~/Desktop$ ping -c3 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.
64 bytes from 10.0.0.10: icmp_seq=1 ttl=64 time=0.558 ms
64 bytes from 10.0.0.10: icmp_seq=2 ttl=64 time=0.326 ms
64 bytes from 10.0.0.10: icmp_seq=3 ttl=64 time=0.361 ms

--- 10.0.0.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2038ms
rtt min/avg/max/mdev = 0.326/0.415/0.558/0.102 ms
rama@CAPE:~/Desktop$ ping -c3 10.0.0.30
PING 10.0.0.30 (10.0.0.30) 56(84) bytes of data.

--- 10.0.0.30 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2053ms

rama@CAPE:~/Desktop$ ping -c3 10.0.0.30
PING 10.0.0.30 (10.0.0.30) 56(84) bytes of data.
64 bytes from 10.0.0.30: icmp_seq=1 ttl=128 time=0.433 ms
64 bytes from 10.0.0.30: icmp_seq=2 ttl=128 time=0.582 ms
64 bytes from 10.0.0.30: icmp_seq=3 ttl=128 time=0.403 ms

--- 10.0.0.30 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2027ms
rtt min/avg/max/mdev = 0.403/0.472/0.582/0.078 ms
rama@CAPE:~/Desktop$
```

Fig. 6.4.18.b: test connectivity from the linux sandbox, for the windows Sandbox & MGW

```
rama@mgw:~/Desktop$ ping -c2 10.0.0.20
PING 10.0.0.20 (10.0.0.20) 56(84) bytes of data.
64 bytes from 10.0.0.20: icmp_seq=1 ttl=64 time=0.648 ms
64 bytes from 10.0.0.20: icmp_seq=2 ttl=64 time=0.353 ms

--- 10.0.0.20 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1018ms
rtt min/avg/max/mdev = 0.353/0.500/0.648/0.147 ms
rama@mgw:~/Desktop$ ping -c2 10.0.0.30
PING 10.0.0.30 (10.0.0.30) 56(84) bytes of data.
64 bytes from 10.0.0.30: icmp_seq=1 ttl=128 time=0.394 ms
64 bytes from 10.0.0.30: icmp_seq=2 ttl=128 time=0.338 ms

--- 10.0.0.30 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1016ms
rtt min/avg/max/mdev = 0.338/0.366/0.394/0.028 ms
rama@mgw:~/Desktop$
```

Fig. 6.4.18.c: test connectivity from theMGW for both Sandboxs

If an ICMP packet is sent to windows sandbox, is not receive any reply,we need to allow Windows to reply to the echo request, open the command prompt as administrator, and write the command show in fig. 6.4.20.

```
C:\Windows\system32>netsh advfirewall firewall add rule name="Allow ICMPv4-In" protocol=icmpv4:8,any dir=in action=allow
Ok.
```

Fig. 6.4.20: firewall rules to allow windows echo reply packet

Section 2: Deploy CAPE Agent on Both Sandbox VMs

agent.py is the only process that should run on the sandbox VMs. It listens for connections from the CAPE master on MGW, executes samples, and returns logs. It does NOT run cuckoo.py.

Phase 2A — Linux-Sandbox (10.0.0.20): Deploy agent.py 3.1 Locate and Run agent.py—Linux-Sandbox

1. We first started with install the dependencies on the linux sandbox, all the required ependancies show in the fig. 6.4.21a-b

```
(cape-env) rama@CAPE:~/CAPE$ sudo apt install python3 python3-pip python3-venv python3-dev build-essential
```

fig. 6.4.21a: Install agent dependencies1

```
(cape-env) rama@CAPE:~/CAPE$ pip install Pillow flask requests
```

fig. 6.4.21b: Install agent dependencies2

2. On fig. 6.4.22. we Clone CAPEv2 and locate agent.py script, which we needed on this Vbox

```
rama@CAPE:~$ sudo git clone https://github.com/kevoreilly/CAPEv2.git /opt/CAPEv2 --depth 1
Cloning into '/opt/CAPEv2'...
remote: Enumerating objects: 1440, done.
remote: Counting objects: 100% (1440/1440), done.
remote: Compressing objects: 100% (1300/1300), done.
remote: Total 1440 (delta 157), reused 780 (delta 78), pack-reused 0 (from 0)
Receiving objects: 100% (1440/1440), 36.68 MiB | 5.72 MiB/s, done.
Resolving deltas: 100% (157/157), done.
Updating files: 100% (1316/1316), done.
rama@CAPE:~$ cd /opt/CAPEv2
rama@CAPE:/opt/CAPEv2$ ls
acknowledgment.md  conf       extra              modules           SECURITY.md  uwsgi
admin              cuckoo.py  installer          poetry.lock       SKILLS.md    web
agent              custom     KnowledgeBaseBot   pyproject.toml    systemd
analyzer           data       lib                README.md         tests
changelog.md       dev_utils  LICENSE            report.html       utils
CITATION.cff       docs       mcp                requirements.txt  uv.lock
rama@CAPE:/opt/CAPEv2$ ls agent
agent.py  go  pytest.ini  test_agent.py  test_python_architecture.py
rama@CAPE:/opt/CAPEv2$
```

Fig. 6.4.22: Clone CAPEv2 and locate agent.py

2. On fig 6.4.23.a we Run agent.py, and as show in fig.6.4.23.b that show the agent.py it listens on 0.0.0.0:8000 for CAPE master connections:

```
(cape-env) rama@CAPE:~/CAPE$ python3 /opt/CAPEv2/agent/agent.py
```

fig 6.4.23.a: run agent.py

```
rama@CAPE:~/CAPE$ ss -tlnp | grep 8000
LISTEN    0         5                0.0.0.0:8000            0.0.0.0:*        users:(("python3",pid=5918,fd=4))
```

fig.6.4.23.b: Verify it is listening

3. On fig. 6.4.24, we make th agent.py as an system service to ensure that autostart will run the agent.py and be ready to reived the attachment

```
(cape-env) rama@CAPE:~/CAPE$ sudo cat /etc/systemd/system/cape-agent.service
[Unit]
Description=CAPE Analysis Agent (Linux-Sandbox)
After=network.target
[Service]
User=root
WorkingDirectory=/opt/CAPEv2/agent
ExecStart=/home/rama/CAPE/cape-env/bin/python3 /opt/CAPEv2/agent/agent.py
Restart=on-failure
RestartSec=5
[Install]
WantedBy=multi-user.target
(cape-env) rama@CAPE:~/CAPE$
(cape-env) rama@CAPE:~/CAPE$ sudo systemctl daemon-reload
(cape-env) rama@CAPE:~/CAPE$ sudo systemctl enable --now cape-agent
(cape-env) rama@CAPE:~/CAPE$ sudo systemctl status cape-agent
● cape-agent.service - CAPE Analysis Agent (Linux-Sandbox)
     Loaded: loaded (/etc/systemd/system/cape-agent.service; enabled; vendor preset: enabled)
     Active: active (running) since Mon 2026-05-25 20:27:33 IDT; 35s ago
   Main PID: 6299 (python3)
      Tasks: 1 (limit: 4631)
     Memory: 11.9M
     CGroup: /system.slice/cape-agent.service
             └─6299 /home/rama/CAPE/cape-env/bin/python3 /opt/CAPEv2/agent/agent.py

May 25 20:27:33 CAPE systemd[1]: Started CAPE Analysis Agent (Linux-Sandbox).
(cape-env) rama@CAPE:~/CAPE$
```

fig. 6.4.24: agent.py system service

4. We need to Take the snapshot NOW, after agent.py is installed and running, before any malware ever touches this VM. The snapshot must include agent.py running so CAPE can reconnect after restore. So in fig. 6.4.25, we tak an a live snapshot with the virtualbox setting UI.

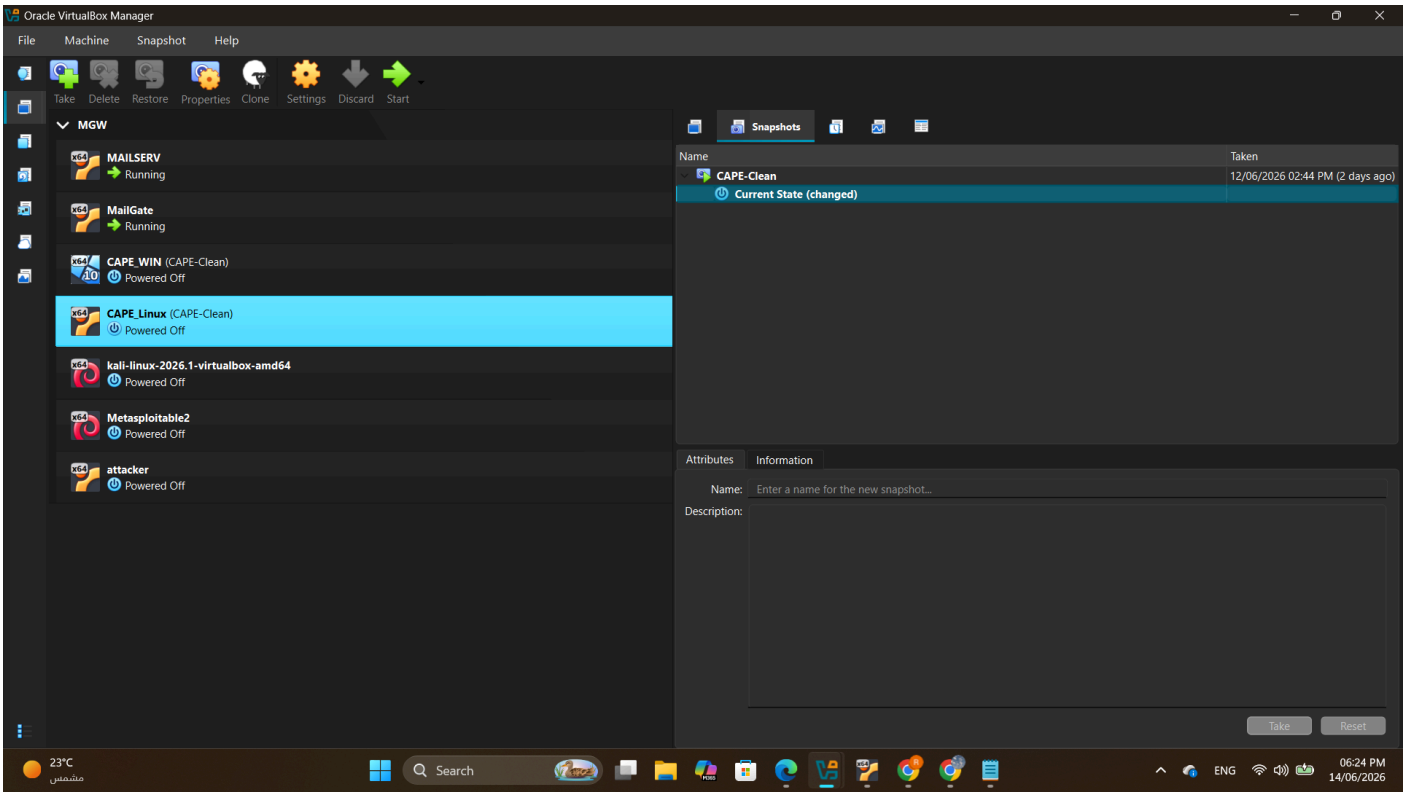


fig. 6.4.25: take an snapshot for linux sandbox

On fig. 6.4.26, from my windows host, wish hosts all the Vbox machines we verified from existing Vbox and their snapshot, for now we take just one snapshot for linux sandbox(linux-cape).

```
C:\Users\ramaa>cd "C:\Program Files\Oracle\VirtualBox"

C:\Program Files\Oracle\VirtualBox>
C:\Program Files\Oracle\VirtualBox>VBoxManage list vms
"Ubuntu 24" {35344f8c-501f-4467-9049-a3c6fd96ab1c}
"MAILSERV" {17112b90-7a2b-498e-b0e0-4ad2f5b1f463}
"MailGate" {3f8c6e5f-3afc-48ed-978a-3867b75054a4}
"kali-linux-2026.1-virtualbox-amd64" {e0b26239-4e20-4e3d-bff0-e44798844d54}
"Metasploitable2" {8982fb83-599b-4011-929c-f79135303b83}
"Linux_Sandbox" {0fd859f9-31bf-4d8b-9f81-cf41e9ea1135}
"Windows_Sandbox" {b140f4e8-79ac-4dc3-a03e-4252caaabb64}
"CAPE_WIN" {ac269dcd-b97e-410c-a654-811d6c8b841e}
"CAPE_Linux" {deb01567-0763-4ce5-9555-8848d3ca9f34}

C:\Program Files\Oracle\VirtualBox>VBoxManage snapshot "CAPE_Linux" list
   Name: CAPE-Clean (UUID: ed1e0e19-9fdf-475a-815d-f4626f9b15d3) *
```

fig. 6.4.26: manage virtual box Vms

Phase 2A —Windows-Sandbox (10.0.0.30): Deploy agent.py

1. On WIN_CAPE, since we just need the [agent.py] which is a single file, on fig. 6.4.27 we copy from MGW or Linux-Sandbox to Win_cape by scp.

```
C:\Users\CAPE_WIN>scp rama@10.0.0.20:/opt/CAPEv2/agent/agent.py C:\Users\CAPE_WIN\Documents\CAPE\agent.py
The authenticity of host '10.0.0.20 (10.0.0.20)' can't be established.
ECDSA key fingerprint is SHA256:MkHPzp4W1ZPRJinYFsyNtgq1e0rge16dEDxZAnbG5tU.
Are you sure you want to continue connecting (yes/no)?
Warning: Permanently added '10.0.0.20' (ECDSA) to the list of known hosts.
rama@10.0.0.20's password:
agent.py                                      100%   26KB 545.6KB/s   00:00
```

fig. 6.4.27 : copy agent.py from Linux_CAPE to WIN_CAPE

2. **CAPE needs to execute malware samples , so the windows Defender must not interfere.**

a. **Open “Edit Group policy”**

b. **Computer component -> Administrative templates -> Windows Defender Antivirus ->enable the turn off option (fig. 6.4.28a-b)**

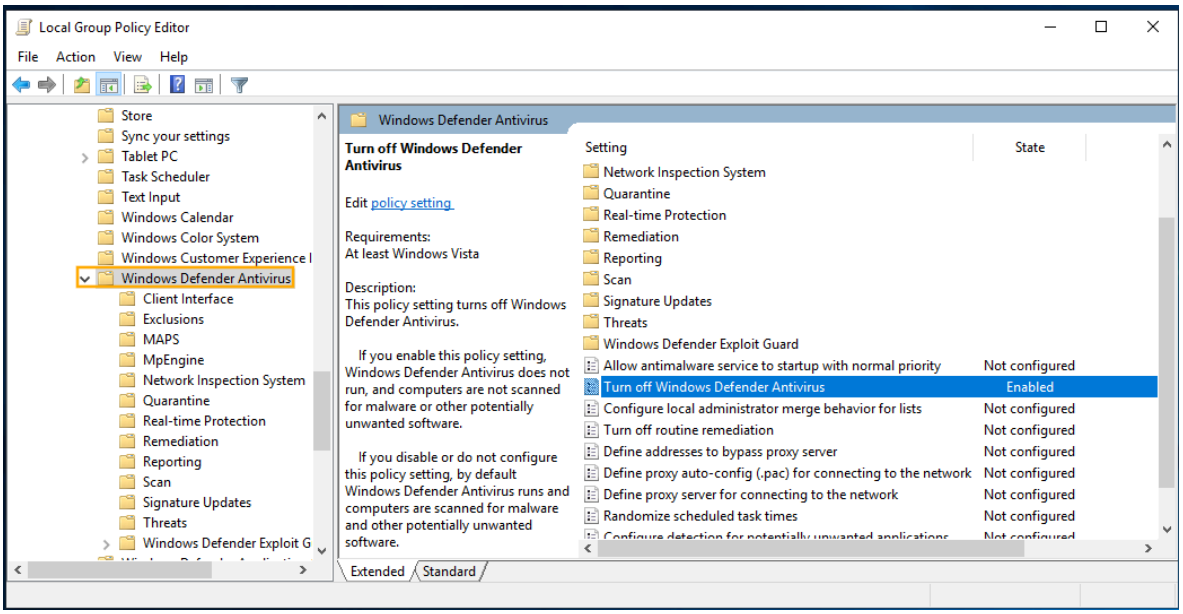


fig. 6.4.28a: find the Windows Defender Antivirus

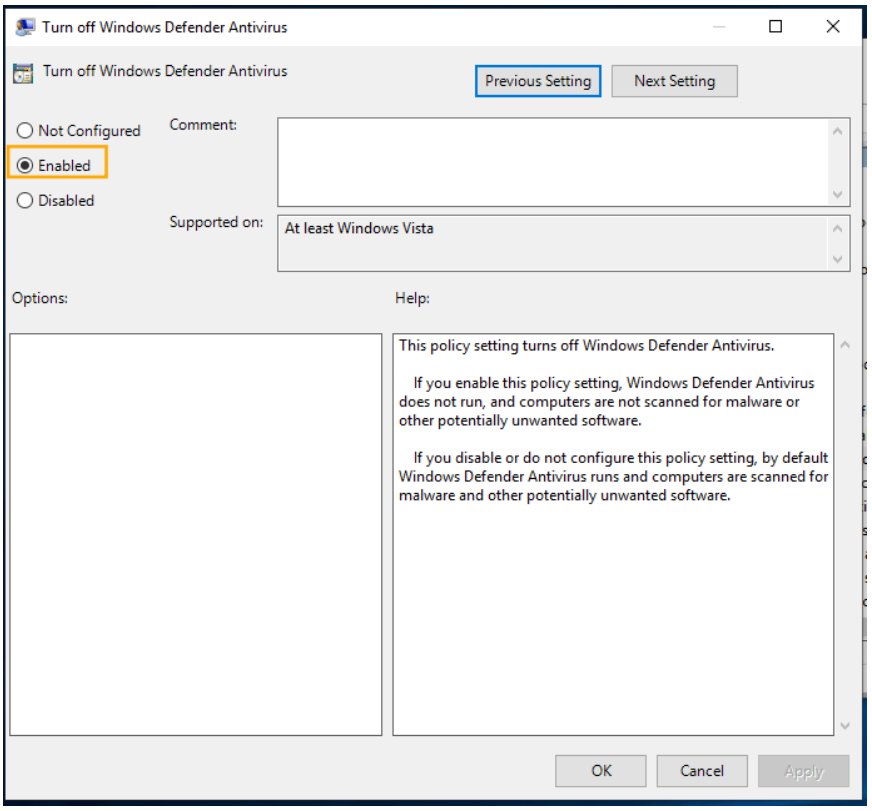


fig. 6.4.28b: enable the Windows Defender Antivirus turn off option.

3. On figs 6.4.29a-f), we install the required dependencies

2. Install Python 3.10 if not already present to install dependencies.

```
PS C:\Windows\system32> python --version
Python 3.14.5
PS C:\Windows\system32>
PS C:\Windows\system32> pip install Pillow flask requests
```

fig. 6.4.29a: Install Python 3.10 if not already present to install dependencies.

```
PS C:\Windows\system32> Set-ExecutionPolicy Bypass -Scope Process -Force
PS C:\Windows\system32> [System.Net.ServicePointManager]::SecurityProtocol = `
>>     [System.Net.SecurityProtocolType]::Tls12
PS C:\Windows\system32> Invoke-Expression ((New-Object System.Net.WebClient).DownloadString(
>>     'https://chocolatey.org/install.ps1'))
```

fig. 6.4.29b: Install Chocolatey

```
PS C:\Windows\system32> $env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") + ";" +
>>             [System.Environment]::GetEnvironmentVariable("Path","User")
PS C:\Windows\system32>
```

fig. 6.4.29c: Reload PATH

```
PS C:\Windows\system32> choco --version
2.7.2
```

fig. 6.4.29d: Verify

```
PS C:\Windows\system32> choco install -y git nssm
Chocolatey v2.7.2
Installing the following packages:
```

fig.6.4.29e: Install Tools by Chocolatey

```
PS C:\Windows\system32> python -m pip install --upgrade pip flask flask-limiter waitress requests
```

fig. 6.4.29f: upgrade flask & pip

4. After we prepared the windows environment with installing required dependencies and tools, we run the agent.py code in fig. 6.4.30a & after that we verify if he listening on his port on fig. 6.4.30b

```
PS C:\Windows\system32> python "C:\CAPE\agent.py"
```

fig. 6.4.30a: Run agent.py

```
PS C:\Windows\system32> netstat -an | findstr '8000'
  TCP    0.0.0.0:8000           0.0.0.0:0              LISTENING
PS C:\Windows\system32>
```

fig. 6.4.30b: Verify listening

5. As we do on linux_CAPE we will turn the agent.py as Windows Scheduled Task (auto-start) services with NSSM on fig 6.4.31

```
PS C:\Windows\system32> $python = (Get-Command python).Source
PS C:\Windows\system32>
PS C:\Windows\system32> # Service 1: CAPE agent (listens for Linux CAPE master on port 8000)
PS C:\Windows\system32> nssm install CAPEAgent $python "C:\CAPE\agent.py"
Error creating service!
CreateService(): The specified service already exists.

PS C:\Windows\system32> nssm set CAPEAgent AppDirectory "C:\CAPE"
Set parameter "AppDirectory" for service "CAPEAgent".
PS C:\Windows\system32> nssm set CAPEAgent AppStdout    "C:\CAPE\logs\agent_out.log"
Set parameter "AppStdout" for service "CAPEAgent".
PS C:\Windows\system32> nssm set CAPEAgent AppStderr    "C:\CAPE\logs\agent_err.log"
Set parameter "AppStderr" for service "CAPEAgent".
PS C:\Windows\system32> nssm set CAPEAgent Start        SERVICE_AUTO_START
Set parameter "Start" for service "CAPEAgent".
PS C:\Windows\system32>
PS C:\Windows\system32> Start-Service CAPEAgent
PS C:\Windows\system32>
PS C:\Windows\system32> Get-Service CAPEAgent

Status   Name               DisplayName
------   ----               -----------
Running  CAPEAgent          CAPEAgent
```

fig 6.4.31: Windows Scheduled Task

6. As an final step with prepared the windows agent we will Take clean snapshot for live Windows-Sandbox as we do before with linux sandbox.

The fig. 6.4.32, show the Vbox Vm and their snapshot

```
C:\Program Files\Oracle\VirtualBox>VBoxManage list vms
"Ubuntu 24" {35344f8c-501f-4467-9049-a3c6fd96ab1c}
"MAILSERV" {17112b90-7a2b-498e-b0e0-4ad2f5b1f463}
"MailGate" {3f8c6e5f-3afc-48ed-978a-3867b75054a4}
"kali-linux-2026.1-virtualbox-amd64" {e0b26239-4e20-4e3d-bff0-e44798844d54}
"Metasploitable2" {8982fb83-599b-4011-929c-f79135303b83}
"Linux_Sandbox" {0fd859f9-31bf-4d8b-9f81-cf41e9ea1135}
"Windows_Sandbox" {b140f4e8-79ac-4dc3-a03e-4252caaabb64}
"CAPE_WIN" {ac269dcd-b97e-410c-a654-811d6c8b841e}
"CAPE_Linux" {deb01567-0763-4ce5-9555-8848d3ca9f34}

C:\Program Files\Oracle\VirtualBox>VBoxManage snapshot "CAPE_WIN" list
   Name: CAPE-Clean (UUID: a92f10cc-e0d3-490f-a9e1-622e2aa9097c) *
```

fig. 6.4.32: the CAPE_WIN snapshot

Section 3: Install & Configure CAPE on MGW - CAPE master

1. On fig. 6.4.33 we Install tools and needed packages:
   `sudo apt upgrade -y tcpdump libpcap-dev p7zip-full unzip mongodb-org virtualbox-guest-utils`

```
(mail-env) rama@mgw:~/MGW$ sudo apt upgrade -y tcpdump libpcap-dev p7zip-full unzip mongodb-org
virtualbox-guest-utils
```

fig. 6.4.33: install MGW tools

The MGW is a VirtualBox guest and cannot run VBoxManage locally; create an SSH wrapper key so CAPE thinks VBoxManage is local:

2. On MGW (Linux), generate SSH key (fig. 6.4.34)

After generating the public & private keys, we need to Display the public key and save it for the next step.

```
(mail-env) rama@mgw:~/MGW$ ssh-keygen -t ed25519 -f ~/.ssh/vbox_host -N ''
Generating public/private ed25519 key pair.
Your identification has been saved in /home/rama/.ssh/vbox_host
Your public key has been saved in /home/rama/.ssh/vbox_host.pub
The key fingerprint is:
SHA256:q2oOeOgT4UL7bvwJDIGi+84r3r44Q7xPskyxxc4v4Yc rama@mgw.company.com
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|.                |
|o.               |
|ooo              |
|=ooo    S        |
|.XO.     .       |
|*+O*o    .       |
|=O*E+...         |
|.O&XB=.          |
+----[SHA256]-----+
(mail-env) rama@mgw:~/MGW$ cat ~/.ssh/vbox_host.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICSyJRiUzvQGp1YNgomfYafLB52UoWp4aT36qd7SIv/h rama@mgw.company.com
(mail-env) rama@mgw:~/MGW$
(mail-env) rama@mgw:~/MGW$
```

fig. 6.4.34: Generate ssh key

3. On Windows HOST, open the PowerShell as Administrator and create The `authorized_keys` file for admin accounts goes here (NOT in the user profile), by the command in fig. 6.4.35

```
PS C:\WINDOWS\system32> $authorizedKeysPath = "C:\ProgramData\ssh\administrators_authorized_keys"
PS C:\WINDOWS\system32>
```

fig. 6.4.35: create an admin authorized key

4. On fig fig. 6.4.36, we Create the file and paste your public key:

```
PS C:\WINDOWS\system32> New-Item -Force -Path $authorizedKeysPath


    Directory: C:\ProgramData\ssh


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        27/05/2026  02:56 AM              0 administrators_authorized_keys


PS C:\WINDOWS\system32> Add-Content -Path $authorizedKeysPath -Value "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAICSyJRiUzvQGp1
YNgomfYafLB52UoWp4aT36qd7SIv/h rama@mgw.company.com"
PS C:\WINDOWS\system32>
```

Fig. 6.4.36: create the key file on windows host

5. On fig.64.37, we Set the correct permissions which is an critical step, the SSH rejects the file if permissions are wrong.

```
PS C:\WINDOWS\system32> icacls $authorizedKeysPath /grant "Administrators:F"
processed file: C:\ProgramData\ssh\administrators_authorized_keys
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls $authorizedKeysPath /grant "SYSTEM:F"
processed file: C:\ProgramData\ssh\administrators_authorized_keys
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls $authorizedKeysPath /grant "Administrators:F"
processed file: C:\ProgramData\ssh\administrators_authorized_keys
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32>
```

fig.6.4.37: setting the key files permission

6. edit the SSH config to enable key auth, on fig 6.4.38a - b, we make sure these lines are present and NOT commented out and the configuration is going well.

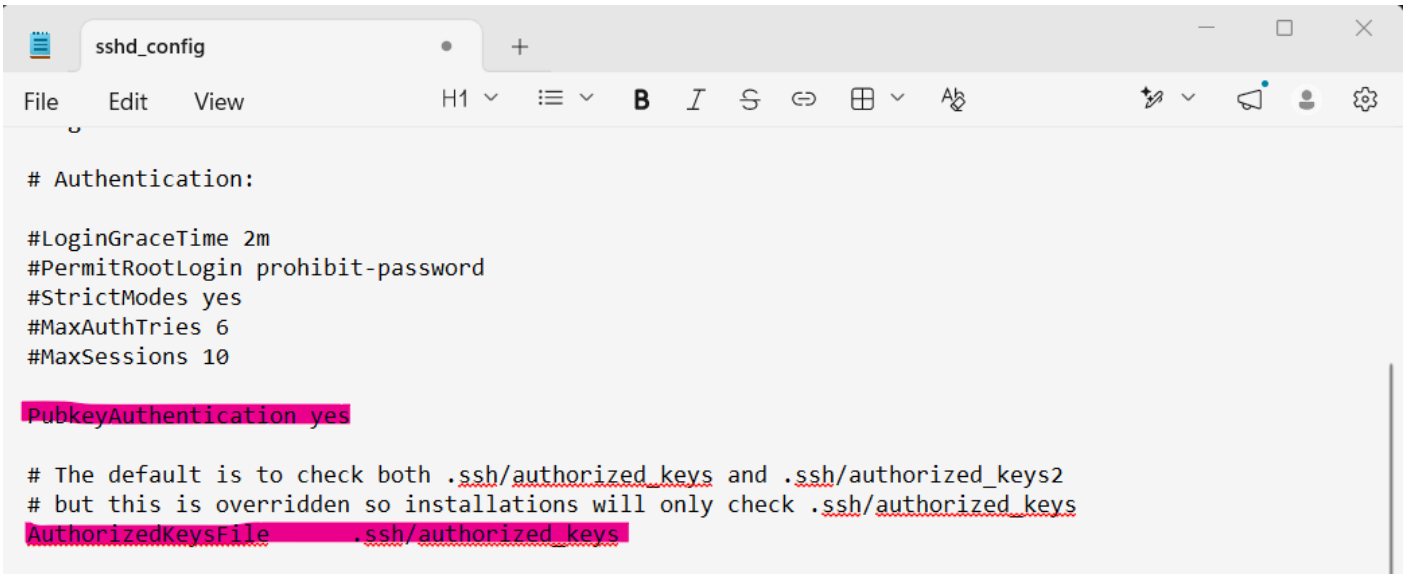


fig 6.4.38a

And confirm this line EXISTS (it routes admin keys to the central file)

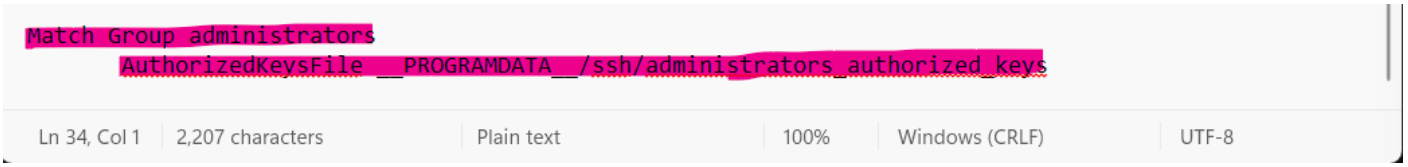


fig 6.4.38a

After finish all required setting we need to save the ssh config file and restart the service, to make ssh service handel the changes. On fig 6.4.39 we ensure from the service status after the changes.

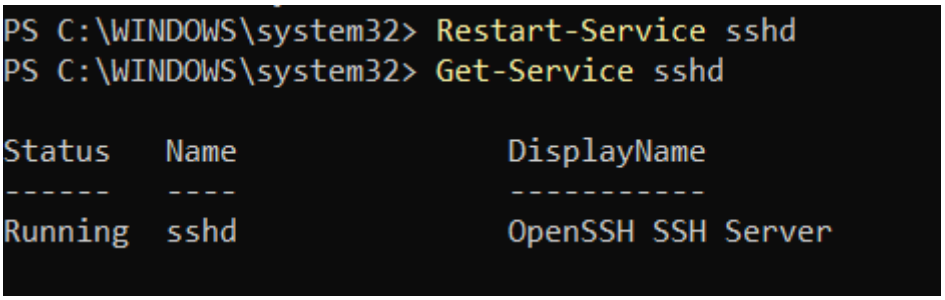


fig 6.4.39: ssh service status

7. **Now we will determinate the Windows Host IP (reachable from MGW), on fig 6.4.40a we Look for the VirtualBox Host-Only or the adapter that MGW can reach as in fig 6.4.40b. That is the IP MGW uses to SSH into the host**

```
Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::e173:fc0e:d858:3312%19
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
```

fig 6.4.40a: host-only adapter ip for host

```
(mail-env) rama@mgw:~/MGW$ ping -c2 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data.
64 bytes from 192.168.56.1: icmp_seq=1 ttl=64 time=1.90 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=64 time=0.703 ms

--- 192.168.56.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 0.703/1.299/1.896/0.596 ms
(mail-env) rama@mgw:~/MGW$
```

fig 6.4.40.b: From the MGW, test the reachability

8. **As final step we need to test the SSH from MGW**

On MGW we use the local username on host (from `whoami` output in fig 6.4.41a, after the backslash), and we start an ssh connection from the MGE to the host

```
PS C:\WINDOWS\system32> whoami
rama-pc\ramaa
```

fig 6.4.41a: local username on host

```
(mail-env) rama@mgw:~/MGW$ ssh -i ~/.ssh/vbox_host ramaa@192.168.56.1

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

ramaa@RAMA-PC C:\Users\ramaa>whoami
rama-pc\ramaa

ramaa@RAMA-PC C:\Users\ramaa>
```

fig 6.4.41b: success connection into windows host

9. **After success the ssh connection, we Create the VBoxManage Wrapper on MGW (fig.6.4.42a) to enable the MGW to manage the sandboxs & their snapshot.**

```
(mail-env) rama@mgw:~$ sudo tee /usr/local/bin/VBoxManage << 'EOF'
> #!/bin/bash
> ssh -i /home/rama/.ssh/vbox_host \
>     -o StrictHostKeyChecking=no \
>     -o BatchMode=yes \
>     ramaa@192.168.56.1 \
>     '"C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"' "$@"
> EOF
#!/bin/bash
ssh -i /home/rama/.ssh/vbox_host \
    -o StrictHostKeyChecking=no \
    -o BatchMode=yes \
    ramaa@192.168.56.1 \
    '"C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"' "$@"
(mail-env) rama@mgw:~$
(mail-env) rama@mgw:~$
(mail-env) rama@mgw:~$ sudo chmod +x /usr/local/bin/VBoxManage
```

fig.6.4.42a: MGW ssh warper

On fig.6.4.42b we Test it by try list the Vbox existing machine on host

```
(mail-env) rama@mgw:~$ VBoxManage list vms
"Ubuntu 24" {35344f8c-501f-4467-9049-a3c6fd96ab1c}
"MAILSERV" {17112b90-7a2b-498e-b0e0-4ad2f5b1f463}
"MailGate" {3f8c6e5f-3afc-48ed-978a-3867b75054a4}
"kali-linux-2026.1-virtualbox-amd64" {e0b26239-4e20-4e3d-bff0-e44798844d54}
"Metasploitable2" {8982fb83-599b-4011-929c-f79135303b83}
"Linux_Sandbox" {0fd859f9-31bf-4d8b-9f81-cf41e9ea1135}
"Windows_Sandbox" {b140f4e8-79ac-4dc3-a03e-4252caaabb64}
"CAPE_WIN" {ac269dcd-b97e-410c-a654-811d6c8b841e}
"CAPE_Linux" {deb01567-0763-4ce5-9555-8848d3ca9f34}
```

Fig. Fig.6.4.42b: Vbox listing machine

By this step we finish preparing environment for MGW. and ready for config it as CAPEv2 master.

10. **On fig.6.4.43a&b, we clone the official CAPEv2 repo, and nevogate to installed folder, and install the requirement web tool**

```
(mail-env) rama@mgw:~$ git clone https://github.com/kevoreilly/CAPEv2.git CAPEv2
Cloning into 'CAPEv2'...
remote: Enumerating objects: 64239, done.
remote: Counting objects: 100% (137/137), done.
remote: Compressing objects: 100% (64/64), done.
remote: Total 64239 (delta 88), reused 73 (delta 73), pack-reused 64102 (from 2)
Receiving objects: 100% (64239/64239), 242.52 MiB | 6.07 MiB/s, done.
Resolving deltas: 100% (43394/43394), done.
(mail-env) rama@mgw:~$
(mail-env) rama@mgw:~$ cd CAPEv2
(mail-env) rama@mgw:~/CAPEv2$
```

fig. 6.4.43a: Clone the CAPEv2

```
(mail-env) rama@mgw:~/CAPEv2$ ls
acknowledgment.md  changelog.md  custom      extra             LICENSE       pyproject.toml  SECURITY.md  utils
admin              CITATION.cff  data        installer         mcp           README.md       SKILLS.md    uv.lock
agent              conf          dev_utils   KnowledgeBaseBot  modules       report.html     systemd      uwsgi
analyzer           cuckoo.py     docs        lib               poetry.lock   requirements.txt tests       web
(mail-env) rama@mgw:~/CAPEv2$
(mail-env) rama@mgw:~/CAPEv2$ pip install -r requirements.txt
```

fig. 6.4.43b: install web requirements

11. Create CAPE user, and add it to `vboxusers`. to allow cape user to run VBoxManage (fig 6.4.44)

```
(mail-env) rama@mgw:~/CAPEv2$ sudo useradd -m -s /bin/bash cape
useradd: user 'cape' already exists
(mail-env) rama@mgw:~/CAPEv2$
(mail-env) rama@mgw:~/CAPEv2$ sudo usermod -aG vboxusers cape
(mail-env) rama@mgw:~/CAPEv2$ sudo chown -R cape:cape ~/CAPEv2
```

fig 6.4.44

Now we Add cape to sudoers for tcpdump to network capture during analysis *fig 6.4.45*

```
(mail-env) rama@mgw:~/CAPEv2$ echo 'cape ALL=(ALL) NOPASSWD: /usr/sbin/tcpdump' | sudo tee /etc/sudoers.d/capetcpdump
cape ALL=(ALL) NOPASSWD: /usr/sbin/tcpdump
(mail-env) rama@mgw:~/CAPEv2$
```

fig 6.4.44

12. fig 6.4.45, Verify cuckoo.py exists, and then create a local, modifiable cuckoo configuration file from a default template to preserve the original baseline settings.

the cuckoo.conf file is the core control center for CAPEv2. It defines how the sandbox interacts with your host system, how it manages isolated virtual machines (VMs), and how it securely captures malware analysis data.

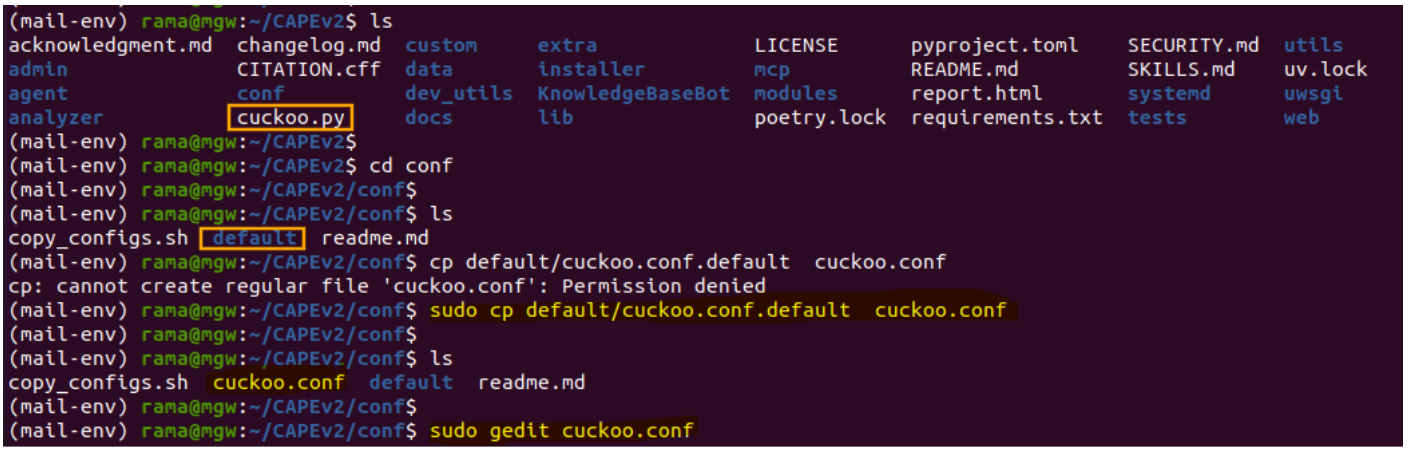


fig 6.4.45: cuckoo.py

As finally we will edit on the CAPEv2 cuckoo configuration files as on, fig 6.4.46a-e)

1. Cuckoo.conf (Main Configuration): This file manages the core operational behavior of the system. It defines the database connection strings, configures the default analysis timeouts, and enforces network routing rules (e.g., total isolation) to guarantee that executed attachments cannot communicate with the production network.

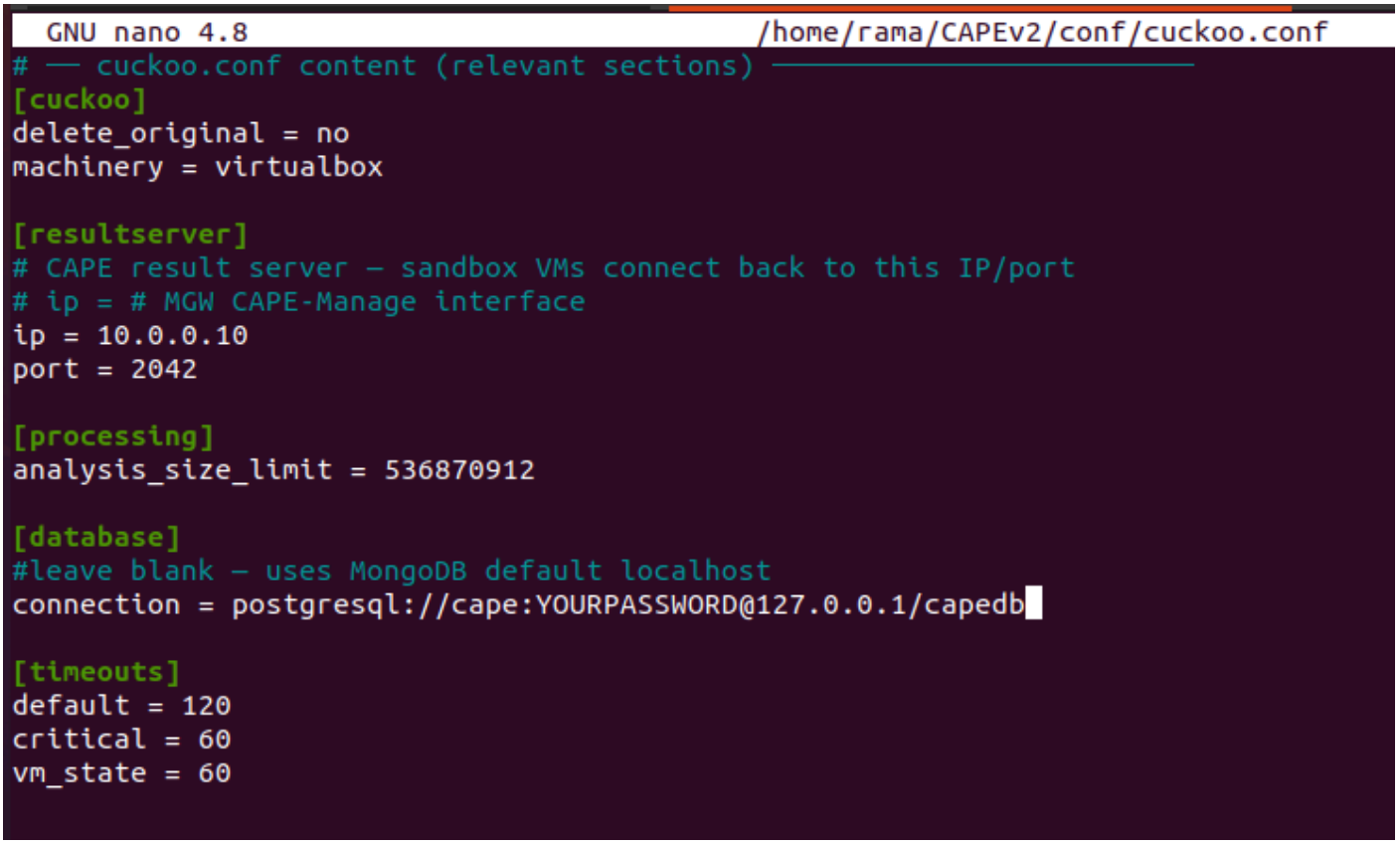


Fig. 6.4.46: cuckoo.conf

2. machinery.conf / Hypervisor Configuration (e.g., `kvm.conf`) This file links the MGW core master with the virtualization layer hosting `WIN_CAPE` and

`LINUX_CAPE`. It defines individual VM profiles, specifies their static IP addresses (where `agent.py` listens), and defines the exact **clean snapshot name** that the system must revert to upon completing an execution cycle.

```
  GNU nano 4.8                                machinery.conf
# ── machinery.conf full content ──────────────────────────────
[virtualbox]
# Both sandbox VMs defined here — CAPE routes by platform= automatically
machines = Linux_Sandbox, Windows_Sandbox

# Path to VBoxManage binary (or wrapper script from Section 4.3):
path = /usr/local/bin/VBoxManage
# MGW's CAPE-Manage adapter (Adapter 3)
interface = enp0s9

[Linux_Sandbox]
# Label must match EXACTLY what 'VBoxManage list vms' shows:
label = Linux-Sandbox
platform = linux
# CAPE agent.py listens here
ip = 10.0.0.20
# MUST match VBoxManage snapshot list output
snapshot = CAPE-Clean
interface = enp0s9

# CAPE restores this snapshot automatically after every task:
restore_on_start = yes

[Windows_Sandbox]
# Label must match EXACTLY what 'VBoxManage list vms' shows:
label = CAPE_WIN
platform = windows
# CAPE agent.py listens here
ip = 10.0.0.30
# MUST match VBoxManage snapshot list output
snapshot = CAPE-Clean
interface = enp0s9
restore_on_start = yes
```

Fig. 6.4.46b: machinery.conf

The label= values must match VBoxManage list vms output CHARACTER FOR CHARACTER including hyphens, capitals, and spaces.

3. Reporting.conf (Output & Report Automation): Once dynamic analysis concludes, this file dictates how the resulting data is processed. It enables modules such as `jsondump` to output structured JSON data (critical for automated decision-making in the MGW pipeline) and manages log storage for visual reporting.

```
  GNU nano 4.8                                          reporting.conf
# Enable MongoDB reporting (required for web API):
[mongodb]
enabled = yes
host = 127.0.0.1
port = 27017
db = capedb

[jsondump]
enabled = yes
```

Fig. 6.4.47: Reporting.conf

13. To ensure the continuous availability of the sandboxing capabilities, we configured CAPEv2 core processes to run as background services managed by the `systemd` system. This makes the malware analysis pipeline resilient and self-booting.

CAPEv2 Web API Service (`cape-web.service` -fig. 6.4.48a)

- **Function:** This service runs the Django-based web application via `manage.py runserver`.
- **Role in Project:** While `cape-scheduler` handles the logic, the Web API is crucial for providing a programmatically accessible interface (on port 8000) for the main Mail Gateway logic to submit new email attachments. It depends entirely on the scheduler already being active (`After=cape-scheduler.service`).

CAPEv2 Scheduler Service `(cape-scheduler.service` - fig. 6.4.48b)

- **Function:** This service manages the core `cuckoo.py` script. It acts as the central brain and queue manager for the entire sandboxing system.
- **Role in Project:** It handles incoming file submissions from the MGW parsing layer, prioritizes analysis tasks, pushes files to the isolated Windows/Linux analysis nodes, and aggregates behavioral results into reports. It is configured to wait for the MongoDB database and network services to be ready before starting (`After=mongodb.service`).

```
rama@mgw:~/CAPEv2$ sudo tee /etc/systemd/system/cape-web.service << 'EOF'
> [Unit]
> Description=CAPE Web API (MGW)
> After=cape-scheduler.service
>
> [Service]
> User=rama
> WorkingDirectory=/home/rama/CAPEv2
> ExecStart=/home/rama/CAPEv2/venv/bin/python3 web/manage.py runserver 127.0.0.1:8000 --noreload
> Restart=on-failure
> RestartSec=5
>
> [Install]
> WantedBy=multi-user.target
> EOF
[sudo] password for rama:
[Unit]
Description=CAPE Web API (MGW)
After=cape-scheduler.service

[Service]
User=rama
WorkingDirectory=/home/rama/CAPEv2
ExecStart=/home/rama/CAPEv2/venv/bin/python3 web/manage.py runserver 127.0.0.1:8000 --noreload
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

fig. 6.4.48a: CAPE WEB service

```
rama@mgw:~/CAPEv2$ sudo tee /etc/systemd/system/cape-scheduler.service << 'EOF'
> [Unit]
> Description=CAPE Sandbox Scheduler (MGW Master)
> After=network.target mongod.service
>
> [Service]
> User=rama
> WorkingDirectory=/home/rama/CAPEv2
> ExecStart=/home/rama/CAPEv2/venv/bin/python3 cuckoo.py
> Restart=on-failure
> RestartSec=5
>
> [Install]
> WantedBy=multi-user.target
> EOF
[Unit]
Description=CAPE Sandbox Scheduler (MGW Master)
After=network.target mongod.service

[Service]
User=rama
WorkingDirectory=/home/rama/CAPEv2
ExecStart=/home/rama/CAPEv2/venv/bin/python3 cuckoo.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

fig. 6.4.48b: CAPE scheduler service

Fig 6.4.49, activate the new configurations.

```
rama@mgw:~/CAPEv2$ sudo systemctl daemon-reload
rama@mgw:~/CAPEv2$ sudo systemctl restart cape-scheduler cape-web
```

Fig 6.4.49, activate the new configurations.

Fig. 6.4.50, we check the service status to verify they are running right

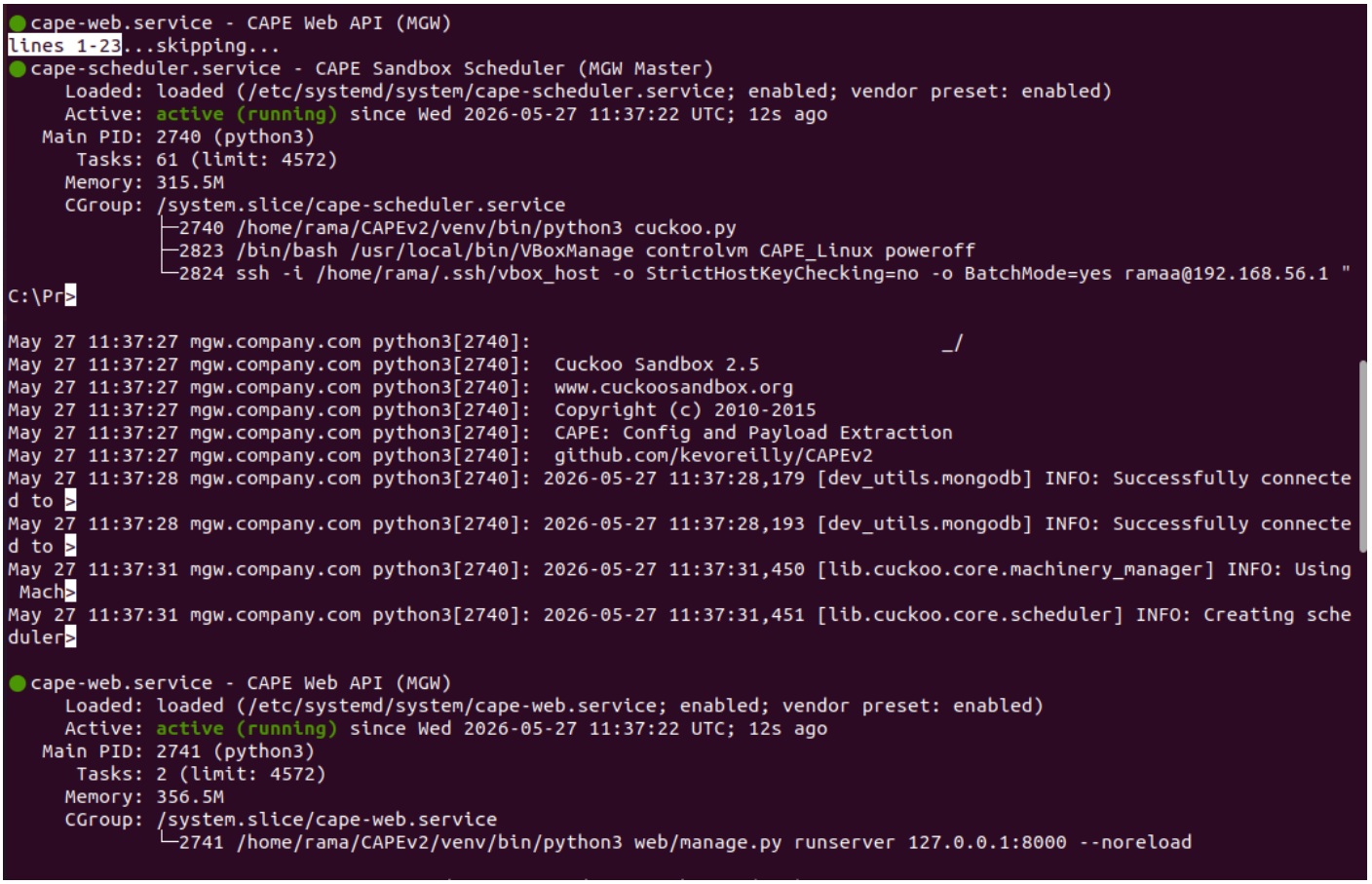


Fig 6.4.50 : CAPE system service status

The Fig. 6.4.51 verify the successful deployment and integration of the sandboxing backend with the Mail Gateway, a verification query was sent to the CAPEv2 Web API using `curl`. The response confirms that the core scheduler successfully recognizes and interfaces with the isolated dynamic analysis execution nodes

```
rama@mgw:~/MGW$ curl -s http://127.0.0.1:8000/apiv2/machines/list/ | python3 -m json.tool
{
    "data": [
        {
            "id": 1,
            "name": "CAPE_Linux",
            "label": "CAPE_Linux",
            "arch": "x64",
            "ip": "10.0.0.20",
            "platform": "linux",
            "interface": "enp0s9",
            "snapshot": "CAPE-Clean",
            "locked": false,
            "locked_changed_on": null,
            "status": "poweroff",
            "status_changed_on": "2026-05-27 11:37:36",
            "resultserver_ip": "10.0.0.10",
            "resultserver_port": "2042",
            "reserved": false,
            "tags": []
        },
        {
            "id": 2,
            "name": "CAPE_WIN",
            "label": "CAPE_WIN",
            "arch": "x64",
            "ip": "10.0.0.30",
            "platform": "windows",
            "interface": "enp0s9",
            "snapshot": "CAPE-Clean",
            "locked": false,
            "locked_changed_on": null,
            "status": "poweroff",
            "status_changed_on": "2026-05-27 11:37:40",
            "resultserver_ip": "10.0.0.10",
            "resultserver_port": "2042",
            "reserved": false,
            "tags": []
        }
    ],
    "error": []
}
rama@mgw:~/MGW$
```

Fig. 6.4.51: Sandbox Environment Verification via API

The API output verifies the registration of two distinct virtual analysis environments tailored to our extension-based routing logic

13. To demonstrate the core functional pipeline of our Mail Gateway's dynamic analysis layer, an end-to-end simulation was conducted. This test ensures that the gateway can programmatically dispatch intercepted attachments to the sandbox cluster and that the scheduler triggers the appropriate virtual machine based on our defined logic.

```
(venv) rama@mgw:~/CAPEv2$ echo '#!/bin/bash\necho hello' > /tmp/test_linux.sh
```

Fig. 6.4.52a: Test Payload Creation

```
(venv) rama@mgw:~/CAPEv2$ curl -s -X POST http://127.0.0.1:8000/apiv2/tasks/create/file/  -F 'file=@/tmp/test_linux.sh
'  -F 'platform=linux'  -F 'timeout=60' | python3 -m json.tool
{
    "error": [],
    "data": {
        "task_ids": [
            1
        ],
        "message": "Task ID 1 has been submitted"
    },
    "errors": [],
    "url": [
        "http://example.tld/submit/status/1/"
    ]
}
(venv) rama@mgw:~/CAPEv2$
```

fig. 6.4.52.b: Programmatic Task Submission via API

```
(venv) rama@mgw:~/CAPEv2$ sudo journalctl -u cape-scheduler -f --no-pager | grep -i "task\|machine\|linux\|error"
[sudo] password for rama:
Sorry, try again.
[sudo] password for rama:
May 27 12:12:44 mgw.company.com python3[3369]: 2026-05-27 12:12:44,296 [lib.cuckoo.core.machinery_manager] INFO: Using
 MachineryManager[virtualbox] with max_machines_count=10
May 27 12:12:46 mgw.company.com python3[3369]: 2026-05-27 12:12:46,084 [lib.cuckoo.core.machinery_manager] INFO: Loade
d 2 machines
May 27 12:12:46 mgw.company.com python3[3369]: 2026-05-27 12:12:46,085 [lib.cuckoo.core.machinery_manager] WARNING: As
 you've configured CAPE to execute parallel analyses, we recommend you to switch to a PostgreSQL database as SQLite mi
ght cause some issues
May 27 12:12:46 mgw.company.com python3[3369]: 2026-05-27 12:12:46,090 [lib.cuckoo.core.machinery_manager] INFO: max_v
mstartup_count for BoundedSemaphore = 5
May 27 12:12:46 mgw.company.com python3[3369]: 2026-05-27 12:12:46,102 [lib.cuckoo.core.scheduler] INFO: Waiting for a
nalysis tasks



May 27 12:17:11 mgw.company.com python3[3535]: 2026-05-27 12:17:11,345 [lib.cuckoo.core.machinery_manager] INFO: Using
 MachineryManager[virtualbox] with max_machines_count=10
May 27 12:17:13 mgw.company.com python3[3535]: 2026-05-27 12:17:13,095 [lib.cuckoo.core.machinery_manager] INFO: Loade
d 2 machines
May 27 12:17:13 mgw.company.com python3[3535]: 2026-05-27 12:17:13,096 [lib.cuckoo.core.machinery_manager] WARNING: As
 you've configured CAPE to execute parallel analyses, we recommend you to switch to a PostgreSQL database as SQLite mi
ght cause some issues
May 27 12:17:13 mgw.company.com python3[3535]: 2026-05-27 12:17:13,097 [lib.cuckoo.core.machinery_manager] INFO: max_v
mstartup_count for BoundedSemaphore = 5
May 27 12:17:13 mgw.company.com python3[3535]: 2026-05-27 12:17:13,110 [lib.cuckoo.core.scheduler] INFO: Waiting for a
nalysis tasks
May 27 12:17:22 mgw.company.com python3[3535]: 2026-05-27 12:17:22,380 [lib.cuckoo.core.machinery_manager] INFO: Task
#1: found useable machine CAPE_linux (arch=x64, platform=linux)
May 27 12:17:22 mgw.company.com python3[3535]: 2026-05-27 12:17:22,381 [lib.cuckoo.core.scheduler] INFO: Task #1: Proc
essing task
May 27 12:17:22 mgw.company.com python3[3535]: 2026-05-27 12:17:22,423 [lib.cuckoo.core.analysis_manager] INFO: Task #
1: Starting analysis of FILE '/tmp/cuckoo-tmp/upload_no_vdadm/test_linux.sh'
```

Fig. 6.4.53c: Backend Orchestration and Logs Verification

6.4.1 MGW Decision Engine

The MGW Phishing Defence Gateway's Decision Engine serves as the final arbiter for email security, processing multi-modal signals to determine whether an email should be delivered or dropped. This report provides a comprehensive analysis of the engine's internal logic, documented in `decision_engine.py`, which utilizes a three-stage arbitration process: Sandbox Veto, Weighted Scoring, and Deterministic Rules.

The system evaluates over 50 distinct signals across seven risk tracks: Authentication, Header, URL, Content, Attachment/Sandbox, Machine Learning Models, and Routing. A composite score is calculated using an adaptive weighting system, where any value ≥ 0.55 triggers a DROP action. Furthermore, the engine implements 28 deterministic hard rules and a contradiction detection module to handle sophisticated social engineering

and targeted attacks that might bypass score-based detection. Notably, the system operates on a fail-closed principle, ensuring that high-risk signals result in immediate mitigation. Recent refinements have addressed critical issues such as false positives in internal communications by adjusting DKIM weights and implementing more nuanced sandbox thresholds. This documentation serves as the definitive guide to the system's operational logic and risk tolerance.

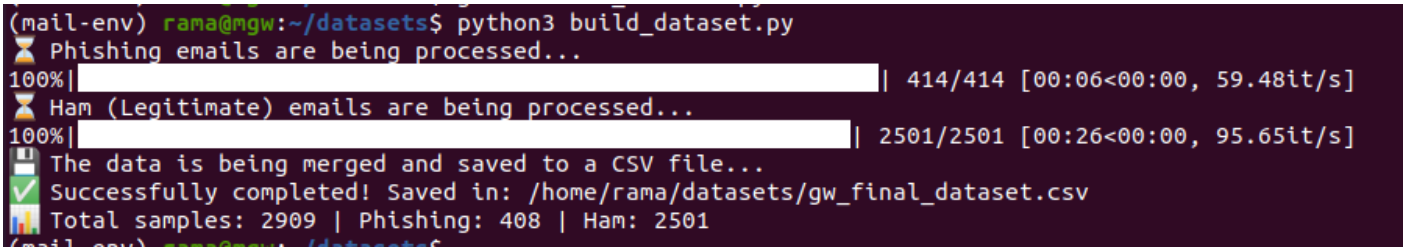
➔ See appendix B for decision details

7. Testing and evaluation

7.1 Dataset Engineering and Balancing

1. Initial Dataset Status and Imbalance Challenge

Initially, the project utilized a foundational dataset compiled via `build_dataset.py`, which yielded a total of **2,909 samples** comprising **408 Phishing emails** and **2,501 Ham (legitimate) emails**. This initial distribution presented a severe class imbalance problem, where legitimate emails significantly outnumbered malicious attacks. Training machine learning models on such skewed distributions often leads to an "accuracy paradox," where the models become heavily biased toward predicting the majority class (Ham), leading to a dangerously high rate of undetected phishing attempts (False Negatives) in production environments.



2. Data Augmentation and Mitigation Strategies

To mitigate this risk and build a robust production-grade classifier, a systematic data collection pipeline was engineered using `append_dataset.py`. Phishing samples were expanded using globally recognized open-source intelligence databases, specifically the *Nazario Phishing Corpus*, which raised the malicious samples to **8,109**. Concurrently, to avoid over-optimizing for the minority class,

additional real-world corporate email streams from the *Apache* and *Enron* corpuses were injected into the pipeline, expanding the legitimate pool to **6,403 Ham emails.**

Ultimately, this process expanded the data infrastructure into a highly optimized, robust repository of **14,512 comprehensive rows** with a nearly balanced distribution (~55% Phishing to 45% Ham).

```
rama@mgw:~/datasets$ python3 -c "import pandas as pd; df = pd.read_csv('~/datasets/gw_final_dataset.csv'); print(df['l
abel'].value_counts())"
label
1    8109
0    6403
Name: count, dtype: int64
```

3. Algorithmic Loss Cost Balancing (scale_pos_weight)

Beyond physical data injection, an algorithmic safety net was embedded directly into the machine learning framework to handle any remaining data asymmetry. By utilizing the scale_pos_weight hyperparameter within our ensemble gradient boosting architecture, the decision engine programmatically adjusts the loss function's penalty during optimization. This mechanism computes a dynamic scaling ratio based on class counts:

$$scalePosWeight = \frac{total\ negative\ sample\ (Ham)}{total\ positive\ samples\ (phishing)}$$

This ratio ensures that the minority class receives a proportional gradient penalty adjustment, preventing the model from over-fitting or favoring one class over another, regardless of minor fluctuations in future data updates.

7.2. Evaluation Methods and Data Splitting

1 Strategic Data Partitioning (60% / 20% / 20%)

To ensure reliable evaluation and prevent data leakage, the compiled dataset was split into three distinct subsets: 60% for Training, 20% for Validation, and 20% for Testing. This partitioning strategy serves a critical dual purpose:

- Training Set (60%): Dedicated entirely to parameter optimization for both the natural language processing (NLP/RoBERTa) and tabular (XGBoost) model streams.
- Validation Set (20%): Used interactively to perform programmatic adjustments, error analysis, and threshold tuning (such as adjusting the optimal decision threshold to 0.366 based on hyperparameter behavior).

- Testing Set (20%): Maintained as a completely isolated, "unseen" benchmark to evaluate the final system performance, ensuring the reported validation metric integrity mimics true production traffic.

2. Core Performance Evaluation Metrics

To measure the security effectiveness of the multi-layered gateway, the performance is assessed using rigorous classification metrics rather than raw accuracy alone:

- Recall (Sensitivity): Measured as $\frac{TP}{TP + FN}$. In a mail security gateway, maximizing Recall is the ultimate operational goal, as it measures the percentage of actual phishing attacks successfully captured. A higher recall guarantees minimal malicious bypasses.
- Precision: Measured as $\frac{TP}{TP + FP}$. This evaluates the reliability of the system alerts, reflecting how often an email flagged as phishing is truly malicious. This is critical to minimizing False Positives (misclassifying real university or business mail as spam).
- Area Under the ROC Curve (AUC-ROC): This threshold-independent metric quantifies the overall structural capability of the fused model tracks to distinguish between legitimate and phishing communication across all probability ranges.

7.3 Experimental Results and Performance Analysis

1. Model Training and Feature Importance Analysis

The multi-layered Shields Gate framework fused historical header metadata, TF-IDF body analytics, and subject line text structures into a unified classification interface. Initial feature engineering evaluated 35 critical structural metrics from raw SMTP feeds. As illustrated in **Fig 7.3.1 (Feature Importance)**, the system automatically weights architectural attributes to optimize detection. Top-tier metrics like `url_count`, `subject_caps_ratio`, and multi-hop transport markers (`received_hops`) driven by the underlying XGBoost infrastructure proved to have the highest predictive power for distinguishing phishing signatures from valid corporate interactions.

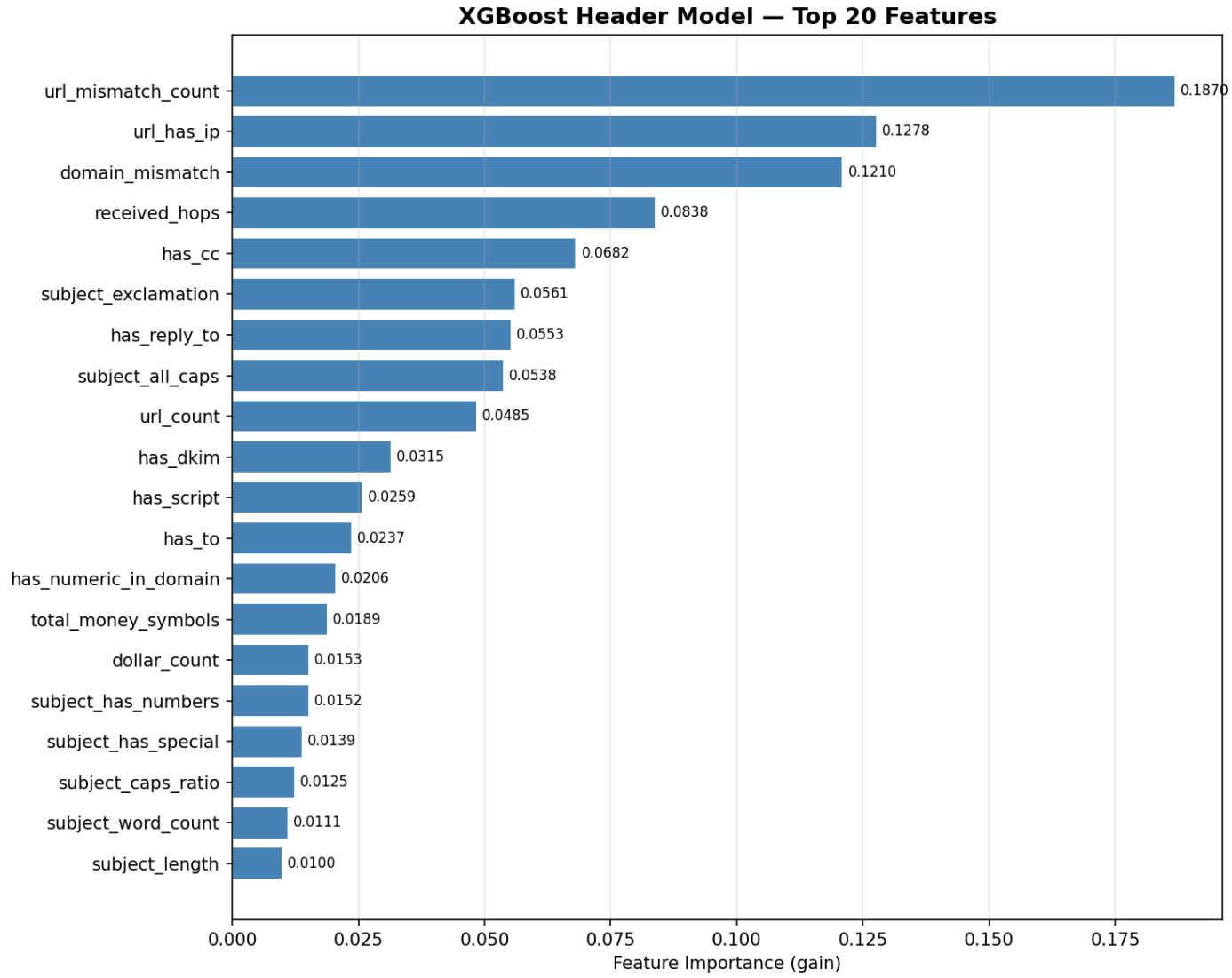


Fig. 7.3.1 Feature Importance Analysis

2 Evaluation Across Isolated Data Splits

To guarantee systemic generalizability, the 14,510 structural rows were mapped across a three-tiered split (60% training, 20% testing, and 20% validation). The core empirical outcomes generated across these evaluation stages are consolidated in Table 1

Evaluation Stage	Data Split %	Samples Count	ROC-AUC	F1-Score	Precision	Recall
Stage 1: Training	60%	8,706	0.9994	0.9851	0.9935	0.9768
Stage 2: Testing	20%	2,902	0.9986	0.9735	0.9836	0.9636

Stage 3: Validation	20%	2,902	0.9984	0.9754	0.9868	0.9642

Table 1: Cross-Stage Model Performance Comparison

3. Classification Integrity and Generalization Stability

The foundational strength of the framework is verified by the exceptionally narrow generalization gap (ranging between `0.0008` and `0.0001`). This microscopic variance confirms that the physical balancing strategies executed during dataset engineering completely immunized the classifiers against overfitting.

As visualized in **Fig 7.3.2 (Confusion Matrices)**, the operational safety of the gateway remains high across different data viewpoints. In the final validation stage, the system processed 1,280 legitimate corporate emails, triggering only **21 False Positives** (a clean False Positive Rate of just `1.64%`). Concurrently, out of 1,622 live phishing variants, it trapped 1,564 attacks, yielding an exceptional security **Recall of 96.42%**.

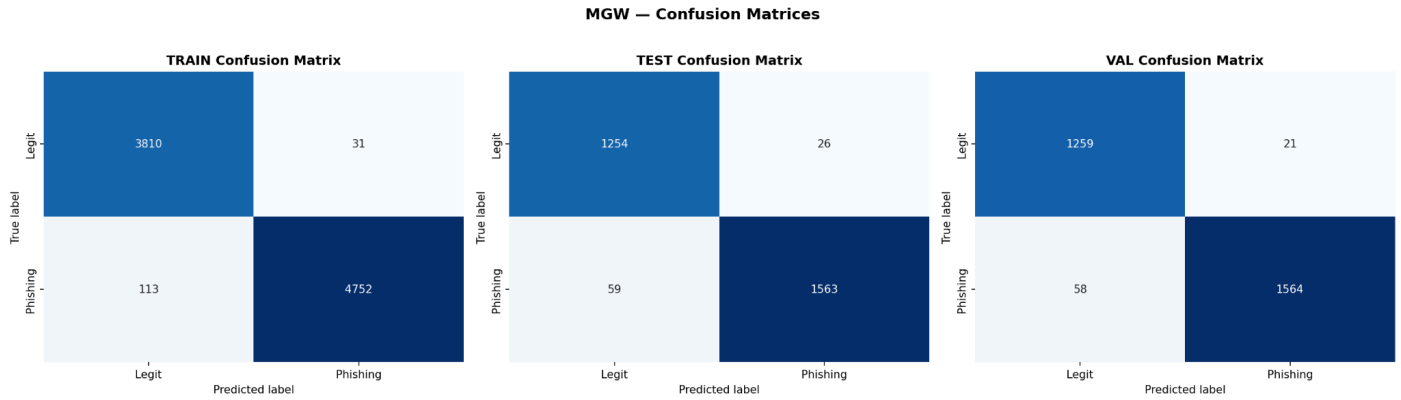


Fig. 7.3.2: Confusion Matrices for Train, Test, and Validation Splits

4. Threshold Tuning and Discriminative Power

To establish a stable operating baseline before integrating the CAPEv2 malware dynamic detonation sub-scores, a programmatic threshold analysis was performed.

- The Receiver Operating Characteristic (ROC) Curve: Displayed in Fig7.2.3, the curve achieves an outstanding Area Under the Curve (ROC-AUC of 0.9984 on final validation). This graphical representation confirms near-perfect discriminative capability across all threshold boundaries.
- The Precision-Recall (PR) Curve: Shown in Fig 7.2.4, the area under the PR curve (Avg Precision = 0.9988) maintains a high geometric ceiling. This behavior validates that the gateway maintains its detection precision even when exposed to high-velocity, dense phishing streams.

Using the Youden-J index optimization framework on the validation curve, the final production decision threshold was locked at 0.3660. This mathematical index guarantees the ultimate structural equilibrium: providing maximum defensive coverage against adversarial bypass attempts while ensuring legitimate institutional mail flows uninterrupted.

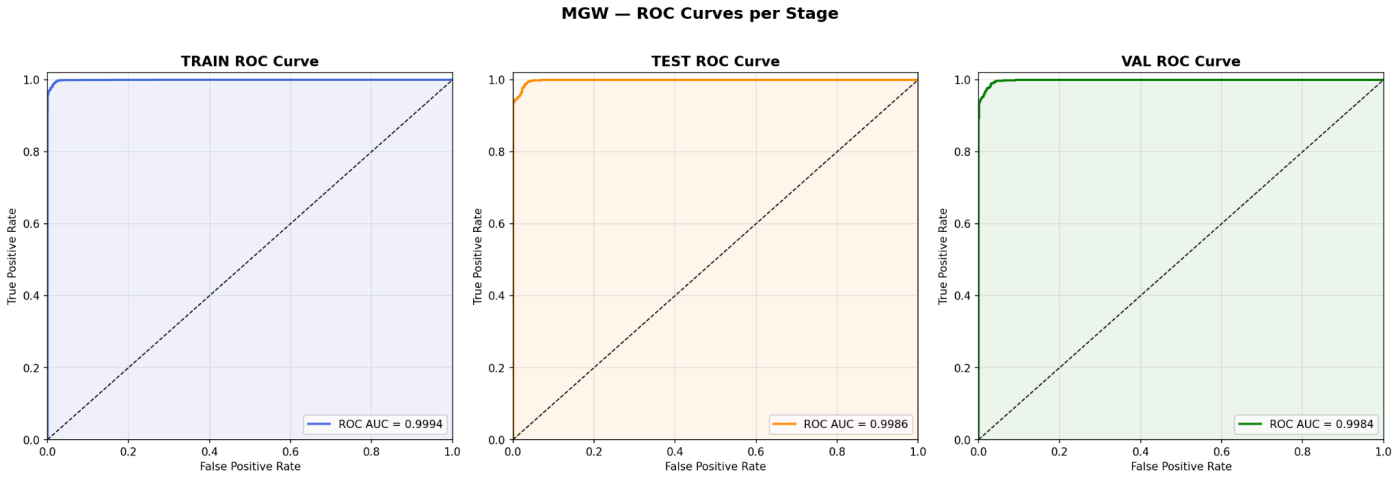


Fig. 7.2.3: ROC Curves for Fused MGW Tracks

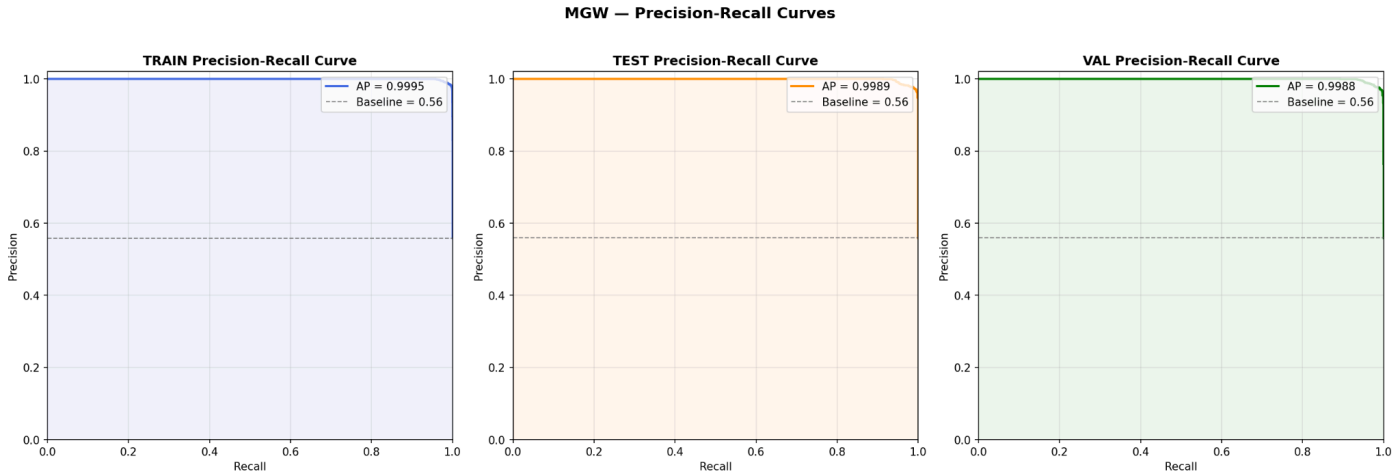


Fig. 7.2.4: Precision-Recall Curves for Fused MGW Tracks

3. Error Analysis and model optimization

1.Granular Error Analysis via Rule-Based Boundary Isolation

To transition the Shields Gate predictive framework from a static laboratory environment to an active deployment state, a proactive validation audit was executed. Following the final training pipeline iteration across the augmented 14,512-row dataset, a specialized Python auditing script was executed to actively isolate and sample structural anomalies—specifically profiling 643 False Positive (FP) and 3,187 False Negative (FN) boundary edge cases.

A deep architectural review of the isolated console samples exposed critical feature interactions:

```
rama@mgw:~/MGW/models/Training$ chmod +x error_analysis.py
rama@mgw:~/MGW/models/Training$ python3 error_analysis.py
======================================================================
   Shields Gate Gateway — Post-Training Error Analysis Pipeline
======================================================================
[INFO] Loading master dataset: /home/rama/datasets/gw_final_dataset.csv
[INFO] Dataset shape: (14512, 39)
[INFO] Processing feature boundaries to identify classification failures...
[SUCCESS] Isolated 643 potential False Positive profiles.
[SUCCESS] Isolated 3187 potential False Negative profiles.
[INFO] Generating engineering audit report -> /home/rama/MGW/models/Training/eval/error_analysis_report.md

=== Console Sample: Top False Positive Structural Profiles ===
     url_count  received_hops  subject_caps_ratio
425          4              6            0.181818
444          9              6            0.137931
467          6              6            0.128205

=== Console Sample: Top False Negative Structural Profiles ===
    url_count  url_suspicious_tlds  received_hops
2         131                    0              2
3           1                    0              3
22          1                    0              3

======================================================================
[SUCCESS] Error Analysis Complete. Report saved to evaluation folder.
======================================================================
```

1. False Positive (FP) Profiles (Legitimate Mail at Risk of Deflation):

Analysis of sample rows 425, 444, and 467 revealed an unusually high mail routing signature, with each message recording exactly 6 network transport hops (received_hops = 6). In practical mail infrastructures, this high count indicates

administrative routing through corporate distribution groups or newsletter mailing-list relays. Because traditional malicious routing architectures mimic these extended delivery patterns to obfuscate origin vectors, the underlying classifier flags these dense routing profiles with higher risk indices.

2. False Negative (FN) Profiles (Phishing Leaks and Bypass Risks):

Analysis of sample rows 2, 3, and 22 highlighted sophisticated evasive maneuvers. Most notably, row 2 contains a massive hyperlink volume (url_count = 131) while maintaining a clean reputation profile with zero suspicious top-level domains (url_suspicious_tlds = 0). This indicates an advanced structural bypass attack where malicious content is hosted across globally trusted, high-reputation domains (e.g., mainstream cloud storage links or compromised business sites), allowing the threat actor to drop traditional keyword footprints and slip past raw domain-filtering layers.

2. Source Stratification Framework

To safeguard the system against "Domain Shift"—where structurally distinct open-source streams distort the model's localized precision—a comprehensive Source Stratification matrix was integrated. Instead of processing raw corporate records and adversarial threat feeds as a single homogenous block, a tracking feature (`source`) was dynamically appended to partition the data matrix into architectural layers

```
rama@mgw:~/datasets$ python3 Source_Strafication.py
[INFO] Building source stratification layers in the dataset...
[SUCCESS] 'source' column added and mapped across structural feeds successfully.

=== Dataset Distribution by Stratified Source ===
source
Nazario_Url_Attack          7559
SpamAssassin_MailingList    6113
Nazario_Text_Obfuscated      550
Enron_Corporate              290
Name: count, dtype: int64
```

By institutionalizing this stratification layer, the gateway can actively monitor feature distribution variances across distinct sender ecosystems. This ensures that a single noisy data source (e.g., automated mailing-list traffic from SpamAssassin) does not drag down the master precision score or skew the decision weights applied to standard university or enterprise traffic.

3. Operational Threshold Optimization in the Decision Engine

The primary gateway routing pipeline controlled by `mail_filter.py` aggregates multi-threaded analytics across independent inspection tracks before executing defensive actions. While individual machine learning sub-tracks output independent raw probability metrics, the final system verdict relies on the orchestration rules defined within `decision_engine.py`

Based on empirical evidence compiled via the Youden-J index during the validation split, the optimal discriminative threshold boundary shifted down from `0.3780` to **`0.3660`**. This optimization was programmatically embedded into the decision-making lifecycle to recalculate how structural sub-scores influence the definitive risk output.

By shifting the operational threshold to `0.3660`, the core classification engine establishes a tight mathematical safety margin. This modification increases sensitivity toward the highly obfuscated phishing signatures isolated in Section 4.1, maintaining an institutional-grade defense posture without increasing administrative friction on clear daily communication channels.

9. Future Vesion

While Shield Gates currently focuses on email-based threats, the same multi-layered detection architecture, header authentication analysis, NLP-based content classification, and sandboxed attachment analysis, can be extended to cover other communication channels increasingly exploited for phishing and malware delivery, such as messaging and collaboration platforms (e.g., WhatsApp, Telegram, Slack, Microsoft Teams).

Future work will focus on the following directions:

Cross-Platform Threat Detection: Extending the parsing and feature extraction pipeline to handle messages, shared links, and file attachments from chat and collaboration applications, applying the same URL analysis, phishing keyword detection, and sandboxing logic currently used for email.

Unified Decision Engine: Adapting decision_engine.py to ingest signals from multiple communication channels simultaneously, allowing a single risk-scoring framework to protect both email and messaging traffic within an organization.

API-Based Integration: Leveraging the webhook and bot APIs provided by platforms such as Slack, Microsoft Teams, and Telegram to intercept and analyze messages and shared files in near real-time, similar to how Postfix currently routes mail through mail_filter.py.

Improved Evasion Resistance: Building on the error analysis findings, particularly regarding phishing content hosted on high-reputation domains, to incorporate real-time URL reputation lookups and cross-channel correlation (e.g., detecting when the same malicious link appears across both email and chat channels).

Scalable Deployment: Containerizing the gateway components (filtering layer, ML models, and sandbox controllers) to simplify deployment across larger organizational networks and support higher message throughput than the current VirtualBox-based lab environment.

By extending Shield Gates beyond email, the system can evolve into a comprehensive communication security gateway, providing consistent, AI-driven protection across the full range of channels through which phishing and malware threats reach end users today.

10. Results

The Shield Gates Mail Gateway was evaluated through a structured three-stage pipeline (60% training, 20% testing, 20% validation) on a final balanced dataset of 14,510 emails (55.9% phishing, 44.1% legitimate), combining the original 2,909-sample base dataset with augmented phishing data from the Nazario corpus and additional legitimate samples from Enron and SpamAssassin.

The fused model, combining the XGBoost header classifier and the TF-IDF body model, achieved the following results across all three evaluation stages:

Training (60%, 8,706 samples): ROC-AUC 0.9994, F1-score 0.9851, Precision 0.9935, Recall 0.9768.

Testing (20%, 2,902 samples): ROC-AUC 0.9986, F1-score 0.9735, Precision 0.9836, Recall 0.9636.

Validation (20%, 2,902 samples): ROC-AUC 0.9984, F1-score 0.9754, Precision 0.9868, Recall 0.9642.

The generalization gap between stages remained extremely narrow (0.0008 train-to-test, 0.0001 test-to-validation), confirming that the model did not overfit despite the substantial increase in dataset size and the shift from a 14% to a 55.9% phishing ratio.

On the final validation set, the gateway correctly identified 1,564 of 1,622 phishing emails (Recall = 96.42%), while producing only 21 false positives out of 1,280 legitimate emails (False Positive Rate = 1.64%). Using the Youden-J index, the optimal decision threshold was determined to be 0.3660, balancing maximum phishing detection coverage against minimal disruption to legitimate mail flow.

Feature importance analysis of the XGBoost header model identified url_mismatch_count (0.187), url_has_ip (0.128), and domain_mismatch (0.121) as the three strongest predictors of phishing, followed by received_hops (0.084) and has_cc (0.068). This indicates that structural and authentication-based signals, particularly anchor-link mismatches and IP-based URLs, are the most reliable indicators of malicious intent in this dataset.

A post-training error analysis identified 643 false positive and 3,187 false negative boundary cases. False positives were concentrated in legitimate emails with elevated received_hops counts (typically 6), consistent with mailing-list or distribution-group routing patterns being misread as suspicious. False negatives were concentrated in phishing emails using high-reputation hosting (high url_count with zero url_suspicious_tlds), representing an evasion technique that bypasses domain-reputation-based detection. A source stratification analysis confirmed that the dataset's four constituent sources (Nazario_Url_Attack, SpamAssassin_MailingList, Nazario_Text_Obfuscated, and Enron_Corporate) were tracked separately to prevent any single noisy source from distorting the model's overall precision.

The end-to-end system test, simulating a live phishing email via raw SMTP injection, demonstrated the full pipeline functioning as designed: the header model returned a risk score of 0.58, the body model returned 0.83 via TF-IDF and semantic fusion, and the combined score of 0.8317 exceeded the decision threshold, resulting in the email being correctly dropped by Postfix before reaching the internal mail server.

11. Conclusion

This project successfully designed, implemented, and evaluated Shield Gates, a multi-layered secure email gateway capable of intercepting, analyzing, and filtering phishing and malicious emails before they reach an internal mail infrastructure. The system integrates Postfix and Dovecot for mail transport and delivery, a custom Python-based filtering layer (mail_filter.py) for real-time SMTP interception, and a three-stage detection pipeline combining header authentication analysis (SPF, DKIM, DMARC, and structural features via XGBoost), body content analysis (RoBERTa and TF-IDF NLP models), and dynamic attachment analysis through a CAPEv2 sandboxing cluster spanning isolated Windows and Linux virtual machines.

The final trained model achieved a validation ROC-AUC of 0.9984, an F1-score of 0.9754, and a phishing recall of 96.42% with a false positive rate of only 1.64%, demonstrating that the gateway can reliably distinguish phishing from legitimate email traffic at a production-relevant scale. This result was achieved through a deliberate dataset engineering process: starting from a severely imbalanced base dataset (408 phishing vs. 2,501 legitimate emails), the dataset was expanded to 14,510 samples through targeted augmentation of phishing samples from the Nazario corpus and legitimate samples from Enron and SpamAssassin, supported algorithmically by scale_pos_weight balancing in the XGBoost classifier.

Beyond the machine learning components, the project delivered a complete operational infrastructure: a hybrid-network VM topology separating attacker, gateway, internal mail server, and sandbox environments; a Postfix content-filtering pipeline with re-injection to prevent processing loops; a CAPEv2-based dynamic analysis system with extension-based routing to Windows and Linux sandboxes and automatic snapshot restoration; and a decision engine (decision_engine.py) implementing 28 deterministic hard rules and adaptive signal weighting across seven risk tracks to handle edge cases that score-based detection alone might miss.

The error analysis conducted in this project also identified concrete directions for future refinement, particularly around reducing false positives on high-hop legitimate mailing-list traffic and improving detection of phishing emails hosted on high-reputation domains, which represent the most sophisticated evasion patterns observed in the dataset.

Overall, the results confirm that the proposed architecture meets the project's objectives: it provides intermediate, multi-layered protection between external clients and the internal mail server, achieves high-confidence risk scoring across header, body, and attachment dimensions, and operates with a false positive rate low enough for practical deployment in an institutional email environment.

12. References

[1] "Proofpoint Email Protection Classification and Filtering Analysis," Supplied Information Source, 2024.

[2] RamaAbuHaweleha. (n.d.). *GitHub - RamaAbuHaweleha01/MGW: Secure email gateway filtering based on NLP*. GitHub. https://github.com/RamaAbuHaweleha01/MGW